Università degli Studi di Trento

Dipartimento di Psicologia e Scienze Cognitive

Corso di Laurea in Scienze e Tecniche di Psicologia Cognitiva

Corso di

Analisi dei dati con applicazioni informatiche

Statistica descrittiva

Dispensa – Moduli 1, 2 e 3

Ottavia M. Epifania

ottavia.epifania@unitn.it

Nota sul materiale

Parte del materiale di questa dispensa è riadattato dal corso *Test per le organizzazioni* dell'Università degli Studi di Padova, tenuto insieme alla Dott.ssa Margherita Calderan (https://ottaviae.github.io/test-organizzioni-fisppa/).

Licenza

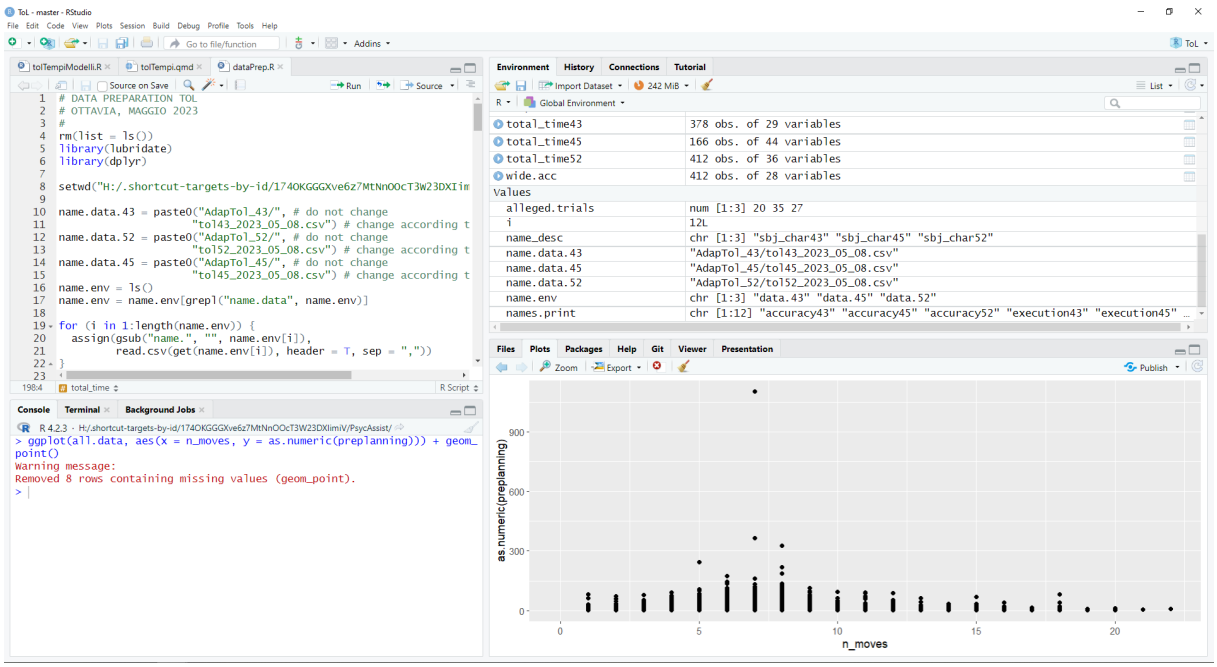
Quest'opera è distribuita con licenza Creative Commons Attribuzione 4.0 Internazionale (CC BY 4.0). Puoi copiare, ridistribuire e modificare il materiale con qualsiasi mezzo e per qualsiasi fine, a condizione di citare l'autrice e di indicare se sono state apportate modifiche.

Per citare questa dispensa:
Epifania, O. M. *Statistica descrittiva. Dispensa del corso Analisi dei dati con applicazioni informatiche, Moduli 1, 2 e 3.* Università degli Studi di Trento.

Indice

1 Modulo 1 – Introduzione a `R`

1.1 RStudio



Ambiente/Environment (Pannello in alto a dx). Contiene tutti gli oggetti che sono stati creati fino a quel momento nell'ambiente di lavoro. Viene automaticamente suddiviso in oggetti di tipo vettoriale o scalare (`values`) e oggetti più complessi come dataframe e liste.

Script. (Pannelli in alto a sinistra). File di tipo testuale con estensione `.R` dove salvare il codice usato per generare gli oggetti presenti nell'ambiente. Si possono combinare codice e commenti (righe che non vengono eseguite e interpretate come codice, sono le righe precedute dal simbolo `#`, vedi Listing **??**). Per eseguire i comandi negli script bisogna usare la combinazione di tasti `ctrl + invio` (per il mac, `cmd + invio`)

Console. (Pannello in basso a sinistra). Pannello di lavoro dove i lavori vengono immediatamente eseguiti immediatamente quando si clicca invio. Le righe di comando non vengon salvate ma sono sempre accessibili premendo con la freccina in su della tastiera. Nella console appaiono anche i risultati dei codici eseguiti nello script.

Plot/Files. (Pannello in basso a dx). Pannnello composto da diversi pannelli. Nel pannello `Plots` compaiono i grafici risultatinti dalle analisi statistiche svolte negli script. Nel pannello `Files` appaiono tutti i file e le cartelle dispoonibili in una data working directory (Vedi Section **??** e Section **??**).

Listing 1.1 Codice con commento

```
# codice per assegnare il numero 25 alla variabile x
x <- 25
```

Console

I comandi nella console vengono eseguiti con click su Invio
Il codice non viene salvato e scompare sequenzialmente.
L'output appare immediatamente nella console.
Per fare lo switch alla console → `ctrl + 2`

Script

Si possono salvare gli script di codice
Per far "correre" qualsiasi comando nello script → Ctrl + Invio (o `cmd + Enter`)
L'output appare nella console.
Per fare lo switch allo script → `ctr + 1`

1.2 Working directories (`wd`)

Working directory. La cartella dentro cui sta lavorando R in quel momento. Diventa di fondamentale importanza perché è dove R si aspetta che ci siano i file (e.g., i dati) con cui volete lavorare

1.2.1 Hard way

Listing 1.2 Funzione per ottenere la wd, ovvero la cartella dentro cui sta lavorando R

```
# per sapere dove sta lavorando R
getwd()
```

```
[1] "C:/Users/Ottavia/Documents/GitHub/stpc-analisi-dati"
```

La `wd` si può cambiare con il comando `setwd()`:

Listing 1.3 Funzione per ottenere cambiare la wd dentro cui sta lavorando R. Dentro la funzione, va messo il percorso completo entro cui si vuole spostare la cartella di lavoro tra virgolette, come nell'esempio sottostante.

```
setwd("C:/Users/Ottavia/Documents/altro-percorso")
```

1.2.2 Easy Way: R Project

Crea automaticamente una working directory (ovvero una cartella) in cui potete trovare tutto il materiale necessario per delle analisi. Questo vi evita di dover pensare alla gestione delle working directories.

Struttura tipica

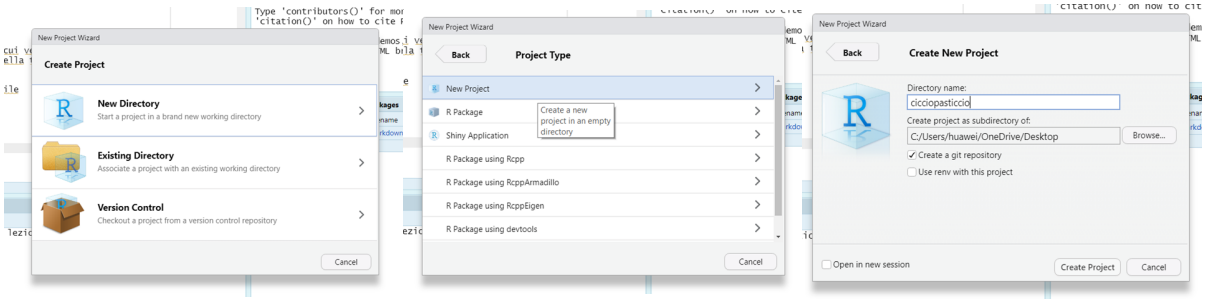
```
my-project/
|
+-- index.qmd
+-- laboratorioTemplate.qmd
+-- myScript.R
+-- my-project.Rproj
+-- data/
|   +-- mydata.csv
```

Nell'esempio precedente, la cartella di ordine superiore è la cartella `my-project`, ovvero la cartella che contiene tutto il vostro progetto. Al suo iinterno si trovano i seguenti file, a diversi livelli:

1. Primo Livello: `index.qmd`, `laboratorioTemplate.qmd`, `myScript.R` (file di quarto con estensione `qmd`, file R con estensione `R`) e una cartella `data`. Il file `my-project.Rproj` è il file del vostro progetto R con estensione `Rproj`.
2. Secondo Livello: all'interno della cartella `data` si trova il file `mydata.csv`. Per accedere a questo file di dati da un file contenuto nella cartella di ordine superiore, bisogna semplicemente comunicare a R che si trova nella cartella data, `"data/mydata.csv"`.

1.2.2.1 Creare un nuovo progetto

Da `File` → New Project e seguire i seguenti step:



(a) Step 1 (b) Step 2 (c) Step 3

Figure 1.1: Creare un progetto R in 3 step

In Figure **??** selezionare l'opzione `New directory`.

In Figure **??** selezionare l'opzione `New Project` (ignorate tutte le altre opzioni).

In Figure **??** date un nome al progetto (e.g., `corso-analisi`) e secgliete la cartella entro cui salvarlo (consigliato: I Documenti del vostro computer).

Figure **??** illsutra il controllo per capire se effettivamente siete dentro al progetto R. Se non siete dentro il progetto R, andate a cercarlo nella cartella dove lo avete creato e fate doppio click sul file con estensione `Rproj`.

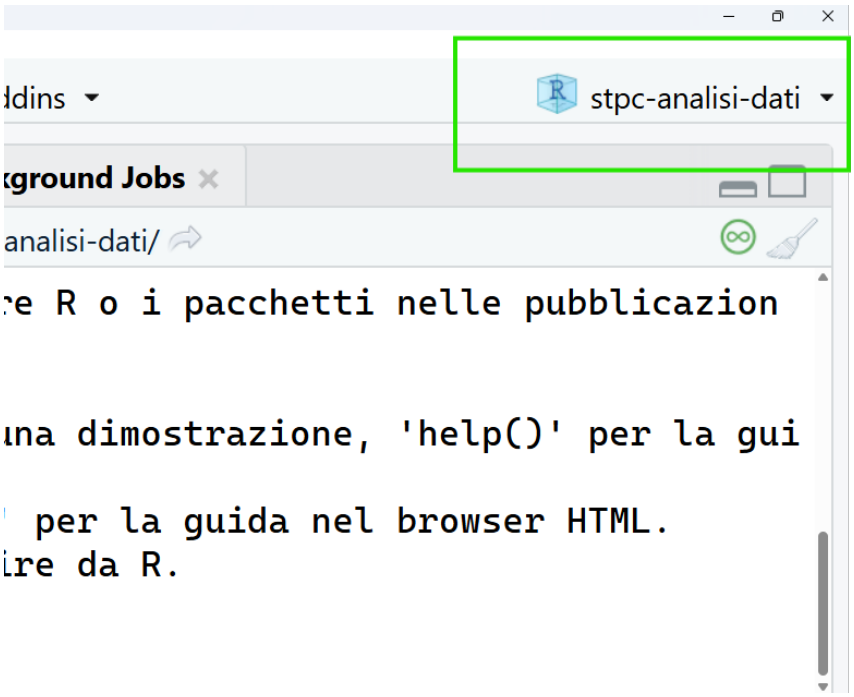


Figure 1.2: Indica il nome del progetto

1.3 Script e reporting

Figure **??** illustra come creare un nuovo script, dove è possibile eseguire e salvare una serie di comandi.

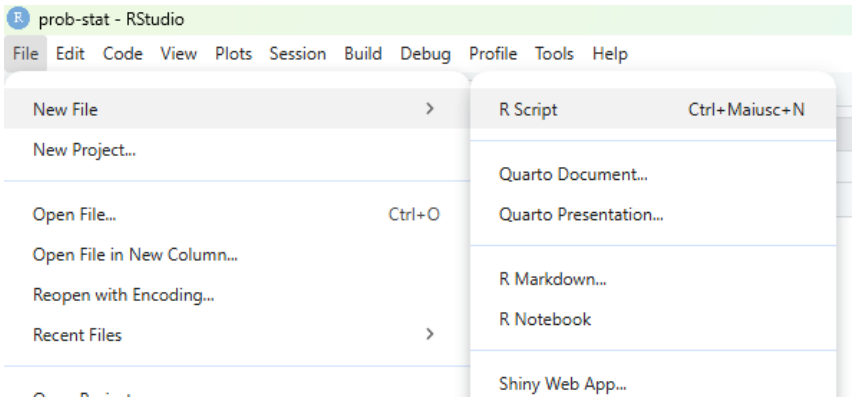


Figure 1.3: Creare un nuovo script

In alternativa, si può usare la sequenza di tasti: `Ctrl + shift + n` (`cmd + shift + n`)

Per salvare uno script, bastare cliccare sull'icona del salvataggio.

1.4 Reporting con quarto

quarto è un markup language integrato con RStudio.

Permette la creazione di documenti (sia pdf sia html) che possono contenere chunk di codice, ovvero delle parti dove viene eseguito il codice R, per ottenere risultati, tabelle e grafici.

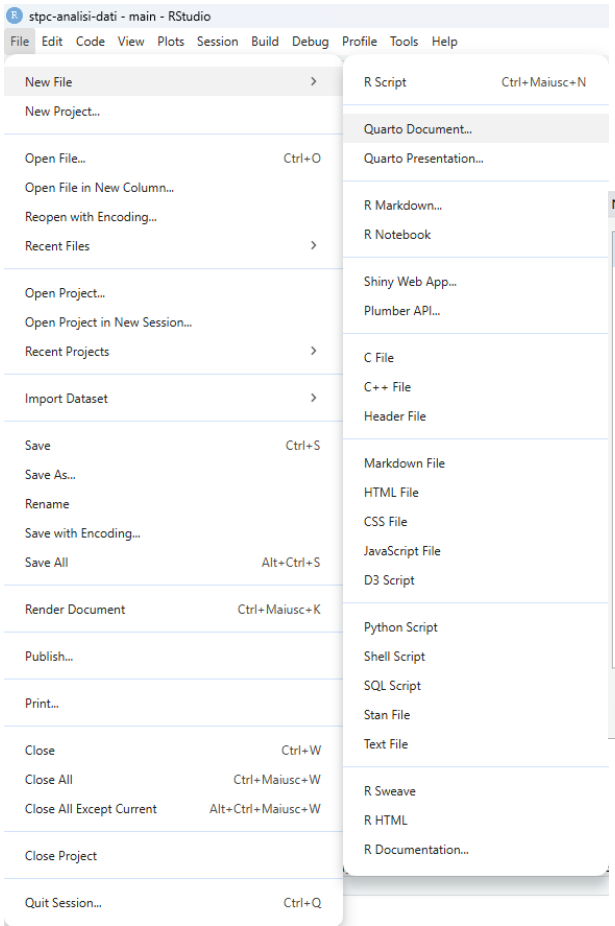
In questo corso, useremo quarto per tutti gli esercizi. Vi verranno forniti dei template, specie all'inizio, nella speranza che riusciate a modificarli e a diventare via via più autonome nel suo utilizzo.

Una parte dell'esame prevede l'utilizzo di quarto per lo svolgimento degli esercizi. le esercitazioni in intinere sono da svolgere su quarto.

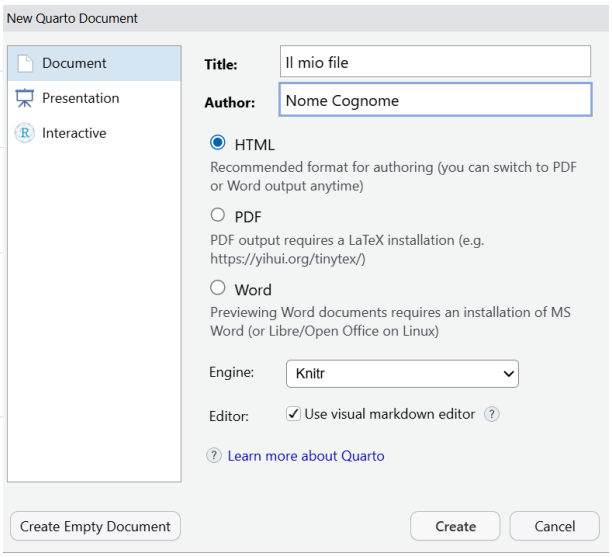
A questo link trovate maggiori informazioni su quarto e una guida veloce sul suo funzionamento.

Figure **??** crea un nuovo file quarto, mentre Figure **??** illustra come specificare varie opzioni del docuemnto (titolo, autrice).

Figure 1.4: Crea un nuovo file quarto



(a) Crea il file



(b) Selezione tipo di file/dati

Figure **??** vi mostra il default che si ottiene quando si crea un documento quarto.

In verde in Figure **??**, lo YAML che controlla l'aspetto generale del documento. Va modificato per cambiare i metadati del documento.

In rosso in Figure **??**, un chunk di codice eseguibile, ovvero una parte del docuemnto che funziona come un qualsaisi script di R e in quanto tale permette di esguire codice.

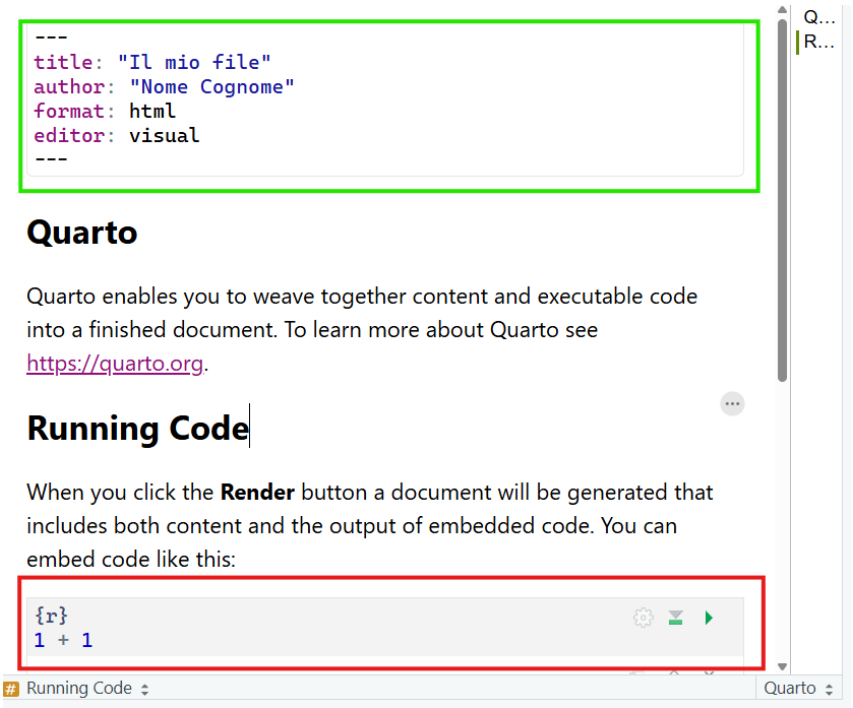


Figure 1.5: Deafult documento di quarto. In verde lo YAML, in rosso un chunk di codice eseguibile come qualsiasi script R

Modifica importante allo YAML

La parte `embed-resources: true` è fondamentale (e va scritta esattamente seguento l'indentazione riportata). Serve a fare in modo che tutti i codici, i risultati e i grafici prodotti nel documento rimangano nel documento quando lo compilate in html.

```
---
title: "Il mio file"
author: "Nome Cognome"
format:
  html:
    embed-resources: true
editor: visual
---
```

Il chunk di codice funziona esattamente come R – è sequenziale, va seguito l'ordine con cui vengono fatte le operazioni

Ogni volta che viene fatto il rendering del documento (`shift + ctrl + k` o freccina azzurra), viene creato un ambiente temporaneo dove viene eseguito il codice.

L'ambiente temporaneo è indipendente dall'ambiente che vedete su R!

Per creare un nuovo chunk: `shift + ctrl + i` oppure Figure **??**:

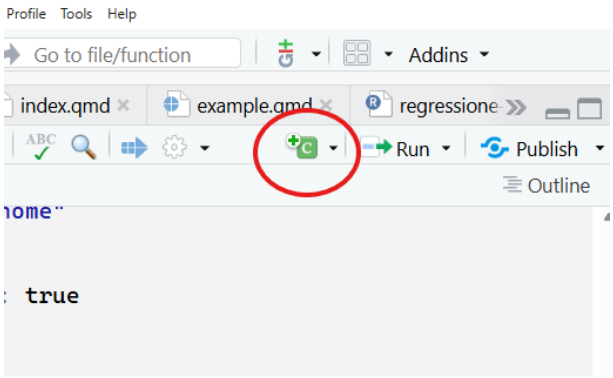


Figure 1.6: Creare un nuovo chunk di codice

1.5 Elementi base

1.5.1 Calcolatrice

```
3 + 2   # più
3 - 2   # meno
3 * 2   # per
3 / 2   # diviso
```

Parentesi e simili:

```
() [] {} "" : ; ,
```

ATTENZIONE

Nei chunk di codice che seguono vedrete spesso l'operatore ; scritto sulla stessa riga di codice dove viene eseguita un'altra operazione, come ad esempio:

```
variabile = 1:5 ; variabile
```

```
[1] 1 2 3 4 5
```

equivale a scrivere

```
variabile = 1:5
variabile
```

```
[1] 1 2 3 4 5
```

1.5.2 Assign (Crea oggetti)

Tutto quello che viene creato in R viene chiamato *oggetto* tramite l'"assegnazione" con specifici simboli, ovvero <- oppure =:

```r
x <- 4
x
```

```
[1] 4
```

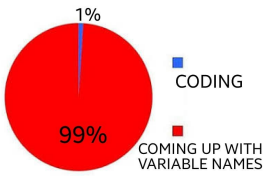
```r
X = 5
X
```

```
[1] 5
```

Gli elementi a destra del simbolo (contenuto) vengono assegnati all'elemento a sinsitra (contenitore) del simbolo

> Attenzione!
>
> R è case sensitive. `x` e `X` sono due oggetti diversi!

1.5.3 Nominare gli oggetti



Nomi validi sono lettere, numeri, punti, underscore (e.g., `variable_name`)

Buoni nomi per le variabili:

- `mean_rt`
- `sum_age`
- `prop_item`

Pessimi nomi per variabili:

- `x`
- `y`
- `ciccio`

Non deve contenere caratteri speciali come #, &, $, ?, etc.

Non deve essere una parola riservata ovvero quelle parole che sono utilizzate da R con un significato speciale (`?reserved` nella console).

```r
.01 = 5
```

```
Error in 0.01 = 5: membro di sinistra dell'assegnazione (do_set) non valido
```

```r
1 = "a"
```

```
Error in 1 = "a": membro di sinistra dell'assegnazione (do_set) non valido
```

```r
TRUE = 567
```

```
Error in TRUE = 567: membro di sinistra dell'assegnazione (do_set) non valido
```

1.5.3.1 Per convenzione

Queste istruzioni sono valide attraverso diversi linguaggi di programmazione.

Fanno riferimento a nomi di oggetti composti da più nomi e/o molto lunghi:

1. `first_data`, ovvero separare i termini con un underscore (e.g., `my_data`)
2. `firstData`, ovvero la prima parola tutta minuscola, la seconda con la prima lettera maiuscola (e.g., `unitnData` invece di `university_of_trento_data`)

1.5.4 L'ambiente

Gli oggetti finiscono nell'ambiente (`environment`) di R.

La funzione `ls()` permette di vedere tutti gli oggetti all'interno dell'ambiente.

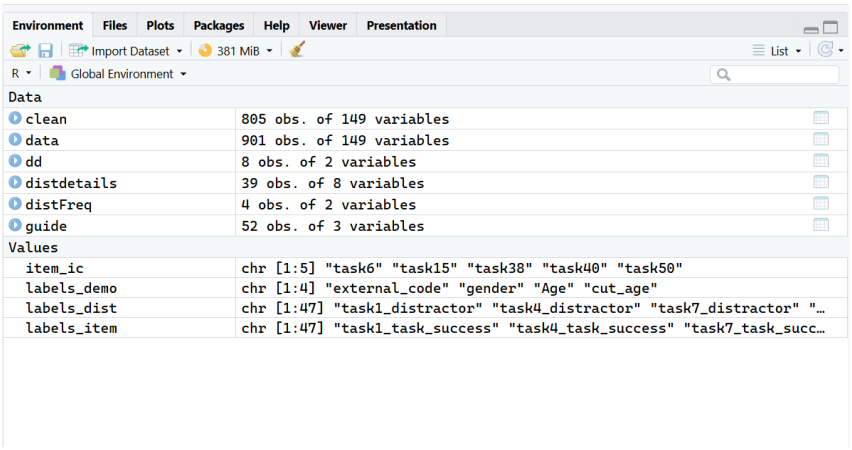


Figure 1.7: Ambiente di R

`rm()` (remove) è la funzione preposta per rimuovere gli oggetti dall'ambiente di R: `rm("oggetto")` (vedi Listing **??**).

Il codice `rm(list = ls())` permette di eliminare tutti gli oggetti nell'ambiente in un colpo solo (equivale alla scopetta).

Listing 1.4 Ottenere gli elementi nell'ambiente e uso della funzione rm

```r
ls() # ottiene tutti gli oggetti nell'ambiente
```

Listing 1.5 Ottenere gli elementi nell'ambiente e uso della funzione rm

```
rm(variabile) # rimuove l'oggetto variabile
ls()
```

```
[1] "variabile" "x"         "X"
```

```
[1] "x" "X"
```

Listing 1.6 Ottenere gli elementi nell'ambiente e uso della funzione rm

```
rm(list = ls()) # rimuove tutto l'ambiente
ls()
```

```
character(0)
```

1.6 Pacchetti e Funzioni

1.6.1 Funzioni

Ogni cosa fatta in R di fatto è riassumibile nelle *operazioni* fatte su *oggetti*

Le operazioni sono le **funzioni**, che permettono di modificare gli oggetti esistenti e di creare di nuovi.

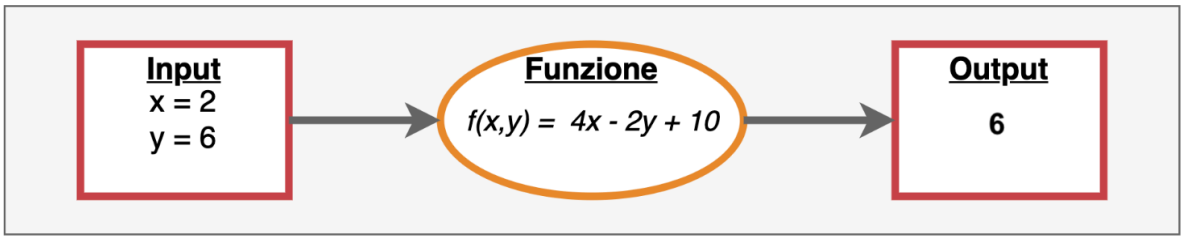
Ad esempio, in Listing **??** viene usata la funzione `sum()` per sommare tutti gli elementi all'interno del vettore. Matematicamente, la funzione `sum()` equivale a $\sum_{i=1}^{k} x_i$ dove k è il totale di elementi x all'interno del vettore X.

Listing 1.7 Applica la funzione somma per sommare tutti gli elementi all'interno di vettore

```
vettore = 1:5
sum(vettore)
```

```
[1] 15
```

Le funzioni in R sono come le funzioni matematiche $f(\cdot)$, dove viene preso un input, viene elaborato secondo le regole della funzione e si ottiene un output:



1.6.1.1 Elementi di una funzione

```
mean(x, trim = 0, na.rm = FALSE, ...)
```

- Nome della funzione: `mean`
- Parentesi che racchiudono gli argomenti della funzione
- argomenti:
  - **obbligatori**, senza valori predefiniti senza di cui la funzione non può funzionare. Nel caso della funzione `mean()`, l'unico argomento obbligatorio è il vettore `x` di valori di cui si vuole calcolare la media
  - **facoltativi** con valori di default (`trim = 0`, `na.rm = FALSE`), che vangono applicati qualora l'utente non ne specifichi di alternativi. Nel caso della media, non viene applicato il trimming dei valori estremi e non vengono tolti i valori NA.

```
mean(vettore)
```

```
[1] 3
```

```
mean(vettore, trim = 0.3)
```

```
[1] 3
```

`?funzione` (Listing **??**) permette di accedere all'help della funzione per scoprire esattamente cosa fa una funzione:

Listing 1.8 Permette di accedere all'help della funzione mean

```
?mean
```

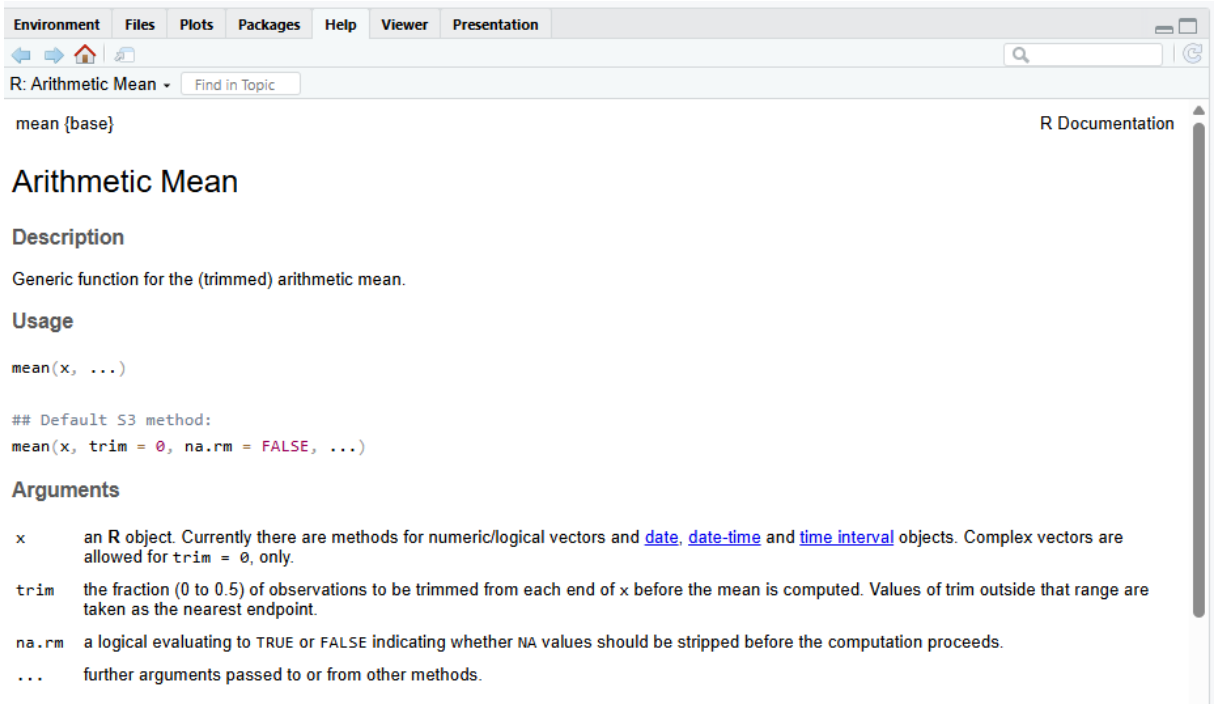


Figure 1.8: Help della funzione mean

1.6.1.2 Installare i pacchetti

Esistono funzioni predefinite all'installazione di R, se ne possono definire di nuove autonomamente, si possono installare dei "pacchetti" dal CRAN (comprehensive R archive network, vedi Listing **??**) o da un file locale, a seconda delle operazioni che si vogliono svolgere.

Listing 1.9 Codice per installare e caricare un pacchetto

```
install.packages("nomePacchetto") # installa (va fatto correre solo una volta)
library(nomePacchetto) # carica il pacchetto (va fatto correre tutte le volte)
```

1.7 Operatori

1.7.1 Operatori Matematici

Tabella **??** illustra alcuni operatori matematici di base, mentre Tabella **??** illustrate le principali funzioni matematiche disponibili in R.

1.7.2 Operatori Relazionali

In R è possibile valutare se una data relazione è vera o falsa. R valuterà le proposizioni e ci restituirà il valore **TRUE** se la proposizione è vera oppure **FALSE** se la proposizione è falsa. Per fare questo si usano gli operatori relazionali in Tabella **??**.

Table 1.1: Operatori matematici in R

Funzione	Cosa fa?	Esempio	Risultato
`+`	addizione	`5.4 + 6.1`	`11.5`
`-`	sottrazione	`9 - 4.3`	`4.7`
`*`	moltiplicazione	`7 * 1.4`	`9.8`
`/`	divisione	`9/3`	`3`
`%%`	resto	`9%%2`	`1`
`^`	potenza	`15 ^ 2`	`225`

Table 1.2: Funzioni matematiche in R

Funzione	Cosa fa?	Esempio	Risultato
`abs()`	valore assoluto	`abs(-8)`	`8`
`sqrt()`	radice quadrata	`sqrt(225)`	`15`
`exp()`	funzione esponenziale	`exp(0)`	`1`
`log()`	logaritmo naturale	`log(1)`	`0`
`round()`	arrotondamento, intero	`round(1.738)`	`2`
`round()`	arrotondamento	`round(1.738, 2)`	`1.74`

Table 1.3: Operatori relazionali in R

Funzione	Nome	Esempio	Risultato
`==`	uguale	`30 == 30`	`TRUE`
`!=`	diverso	`30 != 30`	`FALSE`
`>` / `>=`	maggiore / maggiore o uguale	`30 > 10`	`TRUE`
		`30 >= 10`	`TRUE`
`<` / `<=`	minore / minore o uguale	`30 < 10`	`FALSE`
		`10 <= 10`	`TRUE`
`%in%`	inclusione	`10 %in% c(1, 2, 10)`	`TRUE`

Table 1.4: Operatori logici in R (con x = 30)

Funzione	Nome	Esempio	Risultato
`&`	Congiunzione	`x > 25 & x < 60`	`TRUE`
`|`	Disgiunzione inclusiva	`x > 25 | x > 60`	`TRUE`
`!`	Negazione	`!(x < 18)`	`TRUE`

Table 1.6: Tavole di verità

(a) Congiunzione

a	b	$a \cap b$
V	V	V
V	F	F
F	V	F
F	F	F

(b) Disgiunzione

a	b	$a \cup b$
V	V	V
V	F	V
F	V	V
F	F	F

(c) Negazione

a	$\neg a$
V	F
F	V

(d) Differenza Simmetrica

p	q	$p \oplus q$
V	V	F
V	F	V
F	V	V
F	F	F

1.7.3 Insiemistici

Per capire gli operatori insiemistici (Tabella **??**), definiziamo un oggetto, `x`, come segue:

```
x = 30
```

Gli operatori insiemistici permettono di testare diverse condizioni.

La **congiunzione** testa che siano contemporaneamente vere entrambe le condizioni, ovvero restituisisce un risultato vero se e solo se entrambe le condizioni sono vere.

La **congiunzione** testa che sia vera almeno una delle condizioni, ovvero restituisisce un risultato vero se almeno una delle condizioni è vera.

La **negazione** inverte il valore logico.

1.8 Tavole di verità

Operazione	Simbolo	In R	Significato
Congiunzione (AND)	$A \cap B$	`&`	Vera se entrambe le condizioni sono vere Table **??**.
Disgiunzione (OR)	$A \cup B$	`|`	Vera se almeno una delle due condizioni è vera Table **??**.
Negazione (NOT)	$\overline{A}$	`!`	Inverte il valore logico di A Table **??**.
Differenza simmetrica (XOR)	$A \Delta B$	`xor(A, B)`	Vera se una e una sola delle due condizioni è vera Table **??**.

Lo stesso risultato della funzione `xor()` può essere ottenuto con la combinazione di diversi

1.9 R ed errori

Ci sono diversi livelli di **allerta** quando scriviamo codice:

- **`messaggi`**: la funzione ci restituisce qualcosa che è utile sapere, ma tutto liscio. Appaiono spesso quando si caricano dei pacchetti.
- **`warnings`**: la funzione ci informa di qualcosa di *potenzialmente* problematico, ma (circa) tutto liscio. R ci sta dicendo che è successo qualcosa di insolito e di prestareattenzione, ma non per forza è un problema.
- **`error`**: la funzione non solo ci informa di un **errore** ma le operazioni richieste non sono state eseguite. Solitamente il messaggio di errore ci dice anche cosa sta succedendo.

Come risolvere?

- Capire il messaggio
- Leggere la documentazione della funzione
- Cercare il messaggio su internet
- Chiedere aiuto nei forum dedicati
- Chidere all'IA

1.9.0.1 Esempi di errori

```
mean(myVector)  #
```

```
Error: oggetto 'myVector' non trovato
```

```
dati = read.csv("i-miei-dati.csv")
```

```
Warning in file(file, "rt"): non è possibile aprire il file 'i-miei-dati.csv':
No such file or directory

Error in file(file, "rt"): non è possibile aprire la connessione
```

```
lenght(vettore)
```

```
Error in lenght(vettore): non trovo la funzione "lenght"
```

```
(3,4,5)
```

```
Error in parse(text = input): <text>:1:3: ',' inatteso
1: (3,
      ^
```

1.9.0.2 R e l'uso dell'IA

I prompt devono più specifici possibile e centrati su quello che volete fare e come lo volete fare.

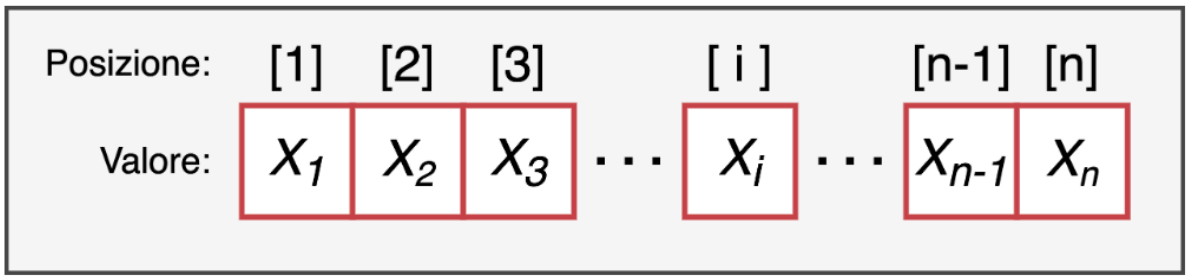
Dovete comunicare chiaramente che volete sempre la soluzioe in R base, possibilmente senza usare funzioni aggiuntive e dovete sempre chiedere che vi spieghi il codice.

Warning

All'esame non potete usare l'IA, ma la potete usare per prepararvi e per aiutarvi nello studio

1.10 Vettori

I vettori sono una struttura dati unidimensionale e sono la più semplice presente in R.



Caratteristiche:

- **la lunghezza**: il numero di elementi da cui è formato il vettore. La funzione `length(x)` permette di ottenere la lunghezza del vettore x, ovvero permette di conoscere il numero di elementi di cui è composto il vettore.
- **la tipologia**: la tipologia di dati da cui è formato il vettore. Un vettore infatti deve esssere formato da **elementi tutti dello stesso tipo** (vedi Section **??**)!

I vettori sono caratterizzati da:

- **un valore**: il valore dell'elemento che può essere di qualsiasi tipo ad esempio un numero o una serie di caratteri.
- **un indice di posizione**: un numero intero positivo che identifica la sua posizione all'interno del vettore.

1.10.1 Creare un vettore con `c()`

Concatena diversi oggetti insieme per creare un nuovo oggetto unico

```
x = c(1, 2, 3) ; x
```

```
[1] 1 2 3
```

```
X = 1:3 ; X
```

```
[1] 1 2 3
```

```
x == X
```

```
[1] TRUE TRUE TRUE
```

1.10.2 Tipologie di vettore

Tipi di oggetti diversi → Tipi di vettori diversi:

- `num` (`numeric`): Numeri `(-0.56, 3, 4.5)`
- `logi` (`logic`): Logico (`TRUE` vs `FALSE`)
- `chr` (`character`): characters `("a", "b", "c")`
- `factor` (`factor`): factor con diversi livelli

La distinzione più importante è tra vettori di tipo categoriale (`chr` e `factor`) e vettori di tipo numerico (`int` e `num`):

- `integer` e `numeric`: Operazioni matematiche E calcolo delle frequenze
- `character` e `factor`: Calcolo delle frequenze

```
months = c(5, 6, 8, 10, 12, 16); months
```

```
[1]  5  6  8 10 12 16
```

```
weight = runif(6, 3, 18) ; weight
```

```
[1] 17.872869 11.524579 15.821762 17.481741  6.328398  7.787142
```

I vettori logici si ottengono testando delle condizioni sulla base dei valori di altri vettori (Listing **??**:

Listing 1.10 Testa se ci sono valori maggior di 12 all'interno del vettore

```
months > 12
```

```
[1] FALSE FALSE FALSE FALSE FALSE  TRUE
```

Il vettore logico risultante può essere utilizzato per filtrare i valori dal vettore orginale (vedi Section **??**).

La funzione `rep()` in Listing **??** crea un vettore di tipo character ripetendo i K valori per X volte. Nell'esempio in Listing **??**, le categorie `low`, `medium`, `high` vengono ripertute tutte per due volte (argomento `each = 2`).

La funzione `factor()` forza il vettore ad essere un vettore con K categorie e un numero di valori uguale al numero di elementi dentro il vettore.

```
[1] low    low    medium medium high   high
Levels: high low medium
```

L'argomento `levels` all'interno della funzione `factor()` (vedi Listing **??**) permette di specificare l'ordine delle categorie, altrimenti il default di R le mette in ordine alfabetico.

Listing 1.11 Creare un vettore di tipo factor usando la funzione rep

```
ses = factor(rep(c("low", "medium", "high"), each = 2))
ses
```

Listing 1.12 Ordinare i livelli della vraibile con levels

```
ses = factor(ses, levels = c("low", "medium", "high"))
```

1.10.3 Indicizzare i vettori

I vttori si indicizzano usando le parentesi quadre:

`nome_vettore[indice]` dove l'indice è la posizione all'interno del vettore che si vuole ottenere.

In R, la numerazione delle posizioni all'interno del vettore va da 1 alla lunghezza del vettore.

Dato ad esempio il vettore `nomi` come segue

```
nomi = c("Pasquale", "Egidio", "Debora", "Luca", "Andrea")
nomi
```

```
[1] "Pasquale" "Egidio"   "Debora"   "Luca"     "Andrea"
```

Possiamo immaginare ogni nome come occupante di una specifica posizione all'interno del vettore, come segue:

Pasquale	Egidio	Debora	Luca	Andrea
1	2	3	4	5

Per cui possiamo estrarre i vari nomi facendo riferimento al loro indice:

`nomi[1]` → Pasquale

`nomi[3]` → Debora

`nomi[seq(2, 5, by = 2)]` → Egidio, Luca

`seq(min, max, by)`

Questa funzione genera un vettore partendo da un valore minimo fino a un valore massimo con uno step incrementale definito dall'utente:

```
seq(1,5, by = 0.5)
```

```
[1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

L'indicizzazione dei vettori funziona anche su vettori di tipo numerico, come negli esempi seguenti:

```r
weight[2] # secondo elemento di weight
```

```
[1] 11.52458
```

```r
(weight[6] = 15.2) # sostituisce il sesto elemento di weight
```

```
[1] 15.2
```

```r
weight[seq(1, 6, by = 2)] # elementi 1, 3, 5
```

```
[1] 17.872869 15.821762  6.328398
```

```r
weight[2:6] # elementi dal 2 al 6 (incluso)
```

```
[1] 11.524579 15.821762 17.481741  6.328398 15.200000
```

```r
weight[-2] # rimuove il secondo elemento
```

```
[1] 17.872869 15.821762 17.481741  6.328398 15.200000
```

I vettori logici possono essere utilizzati per estrarre dteerminati valori da un vettore:

```r
weight > 7
```

```
[1]  TRUE  TRUE  TRUE  TRUE FALSE  TRUE
```

Restituisce un vettore logico dove si `TRUE` se il valore in quella posizione è maggiore di 7 e `FALSE` in ogni altro caso. In questo caso si hanno valori veri (quindi osservazioni con un valore maggiore di 7) nelle seguenti posizioni del vettore:

```
[1] 1 2 3 4 6
```

Usando il vettore logico dentro le parentesi quadre, si otterranno esattamente i valori che occupano quelle posizioni:

```r
weight[weight > 7] # solo i valori > di 7
```

```
[1] 17.87287 11.52458 15.82176 17.48174 15.20000
```

```r
weight[weight >= 4.5 & weight < 8] # valori compresi tra 4.5 e 8
```

```
[1] 6.328398
```

1.11 Matrici

Le matrici sono una struttura dati **bidimensionale** (caratterizzate da 2 dimensioni, la funzione `dim()` permette di sapere da quante righe e da quante colonne è composta) dove il numero di righe rappresenta la dimensione 1 e il numero di colonne la dimensione 2.

```
my_mat = matrix(data = 1:10, nrow = 2, ncol = 5)
my_mat
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    3    5    7    9
[2,]    2    4    6    8   10
```

Per creare una matrice in R si può usare la funzione `matrix()`, come segue

```
`matrix(data, nrow, ncol, byrow = FALSE)`
```

dove `data` sono i valori che si vogliono mettere all'interno della matrice, `nrow` e `ncol` specificano il numero di righe e di colonne che si vogliono mettere nella matrice e `byrow = FALSE` (default) indica che i dati vengono organizzati per colonna (ovvero la matrice viene rimpita seguendo le colonne).

```
# crea una matrice con 3 righe e 4 colonne
# (i valori sono sistemati per riga)
A = matrix(1:12, nrow=3, ncol = 4, byrow = TRUE)
# assegna dei nomi alle righe
rownames(A) = paste("a",1:nrow(A), sep ="_")
# assegna dei nomi alle colonne
colnames(A) = paste("b",1:ncol(A), sep ="_")
A
```

```
    b_1 b_2 b_3 b_4
a_1   1   2   3   4
a_2   5   6   7   8
a_3   9  10  11  12
```

Nel caso precedente, la lunghezza del vettore organizzato nella matrice rientrava perfettamente nel numero di celle disponibili all'interno della matrice stessa. Se questo non succede, si hanno due casi:

Il numero di celle è un moltiplicando/dividendo della lunghezza del vettore

```
data = 1:12
matrix(data, nrow = 6, ncol =4)
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    7    1    7
[2,]    2    8    2    8
```

```
[3,]    3    9    3    9
[4,]    4   10    4   10
[5,]    5   11    5   11
[6,]    6   12    6   12
```

Non succede niente, non appaiono warning e la matrice viene creata ripetendo due volte il vettore originale.

Il numero di celle NON è un moltiplicando della lunghezza del vettore

```
data = 1:12
matrix(data, nrow = 3, ncol =2)
```

```
Warning in matrix(data, nrow = 3, ncol = 2): data length differs from size of
matrix: [12 != 3 x 2]

     [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
```

Si ottiene un warning, ma R comunque crea la matrice come può.

1.11.1 Caratteristiche

Le matrici di fatto sono la concatenazione bidimensionale di una serie di vettori. Di conseguenza:

- Possono contenere una sola tipologia di dati
- Essendo bidimensionali, sono necessari due indici di posizione (righe e colonne) per identificare un elemento
- Possono essere viste come un insieme di singoli vettori

1.11.2 Indicizzare le matrici

La logica per l'indicizzaizone è la stessa che si usa per i vettori, solo che all'interno delle parentesi quadre va inserito anche l'indice per le colonne:

`my_matrix[righe, colonne]`

Il primo indice fa sempre riferimento alle righe, il secondo alle colonne:

Table 1.7: Struttura di una matrice

	[,1]	[,2]	[,3]
[1,]	[1, 1]	[1, 2]	[1, 3]
[2,]	[2, 1]	[2, 2]	[2, 3]
[3,]	[3, 1]	[3, 2]	[3, 3]

Ad esempio, data la matrice `A`:

```
A
```

```
    b_1 b_2 b_3 b_4
a_1   1   2   3   4
a_2   5   6   7   8
a_3   9  10  11  12
```

`A[1, ]` → 1, 2, 3, 4

`A[, 2]` → 2, 6, 10

`A[2, 3]` → 7

1.12 Dataframe

Il dataframe è la struttura più "complessa", utile e potente di R e come le matrici, è composto da due dimensioni (righe × colonne).

A differenza delle matrici, può contenere dati di tipo diverso:

- Ogni elemento (colonna) è un vettore con un nome associato
- Ogni colonna deve avere lo stesso numero di elementi di conseguenza ogni riga ha lo stesso numero di elementi (struttura rettangolare).
- **Tutte le colonne (quindi tutti i vettori contenuti nel dataframe) devono avere la stessa lunghezza**

  data.frame(. . .)

```
sesso = c(rep(c("M","F"), length=length(weight)))
data = data.frame(id = paste0("id", 1:length(weight)),
                  weight, sesso)
data
```

```
   id    weight sesso
1 id1 17.872869     M
2 id2 11.524579     F
3 id3 15.821762     M
4 id4 17.481741     F
5 id5  6.328398     M
6 id6 15.200000     F
```

Per convenzione, i dati sperimentali vengono organizzati in modo che le unità statistiche siano sulle righe e le variabili sulle colonne. In questo modo, sapendo il numero di righe si può ricavare il numero di unità statistiche, sapendo il numero di righe si può ricavare il numero di variabili misurate su quelle unità statistiche.

1.12.1 Indicizzare i dataframe

I dataframe si possono indicizzar come le matrici utilizzando le parentesi quadre e i rispettivi indici di riga e colonna:

```
data[r, c]
```

Si può anche usare un'indicizzazione basata sull'estrazione delle singole colonne con l'operatore $:

```
data$nome_variabile
```

```
data
```

```
   id    weight sesso
1 id1 17.872869     M
2 id2 11.524579     F
3 id3 15.821762     M
4 id4 17.481741     F
5 id5  6.328398     M
6 id6 15.200000     F
```

```
data[2, 1]
```

```
[1] "id2"
```

```
data$weight
```

```
[1] 17.872869 11.524579 15.821762 17.481741  6.328398 15.200000
```

```
data$weight[2]
```

```
[1] 11.52458
```

La stessa logica con cui dai vettori si estraggono tutte le osservazioni che soddisfano un determinato requisito

```
weight[weight >= 7]
```

```
[1] 17.87287 11.52458 15.82176 17.48174 15.20000
```

Nel caso del dataframe:

Estrae tutte le righe che soddisfano la condizione e tutte le colonne.

```r
data[data$weight >=7, ]
```

```
   id   weight sesso
1 id1 17.87287     M
2 id2 11.52458     F
3 id3 15.82176     M
4 id4 17.48174     F
6 id6 15.20000     F
```

Estrare i valori del vettore `data$id` che soddisfano la condizione sulla variabile weight.

```r
data$id[data$weight >=7 & data$weight < 9]
```

```
character(0)
```

1.12.2 Attributi e caratteristiche

Le funzioni **`names()`** , **`dim()`**, **`nrow()`**, **`ncol()`** sono alcune delle funzioni per ottenere informazioni sulle caratteristiche del dataframe.

La funzione più utile è **`str()`** poichè ci restituisce una veloce overview della struttura del dataframe: dimensioni, tipi di variabili.

```r
names(data) # estrae i nomi del dataset
```

```
[1] "id"     "weight" "sesso"
```

```r
dim(data) # numero di righe e numero di colonne del dataset
```

```
[1] 6 3
```

```r
str(data) # restituisce la struttura
```

```
'data.frame':   6 obs. of  3 variables:
 $ id    : chr  "id1" "id2" "id3" "id4" ...
 $ weight: num  17.87 11.52 15.82 17.48 6.33 ...
 $ sesso : chr  "M" "F" "M" "F" ...
```

```r
nrow(data) # numero di righe (equivale al numero di unità statistiche)
```

```
[1] 6
```

```r
ncol(data) # numero di colonne (equivale al numero di variabili)
```

```
[1] 3
```

NB

La funzione **str()** funziona su tutti gli oggetti di R (vettori, matrici, dataframe, liste). Restituisce sempre le informazioni circa la dimensionalità e il numero (e tipo) di elementi contentui nell'oggetto.

1.13 Liste

```
list(name1 = obj1, name2 = obj2, name3 = obj3, . . .)
```

Permette di creare liste di oggetti diversi (matrici, vettori, variabli) di diverso tipo (**numeric**, **character**, **integer**). Ogni oggetto nella lista può avere un nome diverso che può essere utilizzato per l'indicizzazione (vedi Section **??**).

Molti dei risultati ottenuti con le funzioni per le analisi statistiche sono organizzati in liste, proprio perché permettono di contenere oggetti di natura diversa e con dimensionalità diversa.

Listing 1.13 Crea una lista con oggetti diversi (vettori e matrici). Ad ogni oggetto viene assegnato un nome

```
A <- c(1,8,5,0)
M <- matrix(A, 2, 2)
Nomi3 <- c("giorgio", "ugo", "anna")
LV <- c(TRUE, FALSE, TRUE, TRUE)
OgLista <- list(a = A, Mat = M, nomi = Nomi3, X = LV)
str(OgLista)
```

```
List of 4
 $ a   : num [1:4] 1 8 5 0
 $ Mat : num [1:2, 1:2] 1 8 5 0
 $ nomi: chr [1:3] "giorgio" "ugo" "anna"
 $ X   : logi [1:4] TRUE FALSE TRUE TRUE
```

1.13.1 Indicizzare le liste

```
lista[[i]]
```

`[[i]]`: indice i dell'i-esimo oggetto all'interno della lista a cui si vuole accedere (vedi Listing **??**)

Listing 1.14 Accede al secondo oggetto della lista con il suo indice

```
OgLista[[2]]
```

```
     [,1] [,2]
[1,]    1    5
[2,]    8    0
```

`lista$nome_oggetto`

Gli oggetti dentro le liste si possono indicizzare anche attraverso i loro nomi usando l'operatore `$` (vedi Listing **??**):

Listing 1.15 Accede all'oggetto Mat nella lista usando il suo nome

```
OgLista$Mat
```

```
     [,1] [,2]
[1,]    1    5
[2,]    8    0
```

1.14 Importare dati esterni

1.14.1 Importa base

`read.csv2("file_path", header = TRUE, sep = ";")`

Il default della funzione prevede che la prima riga contenga i nomi delle variabili (`header = TRUE`) e che il separatore di colonne (ovvero il valore che fa riconoscere ad R le diverse colonne del dataset) sia ";" (`sep = ";"`).

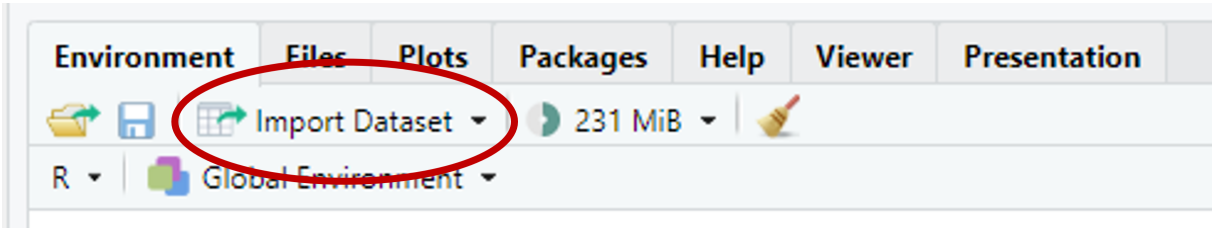
Se si possiede un dataset chiamato "dati" nella cartella "data", questo codice lo importerà all'interno dell'ambiente.

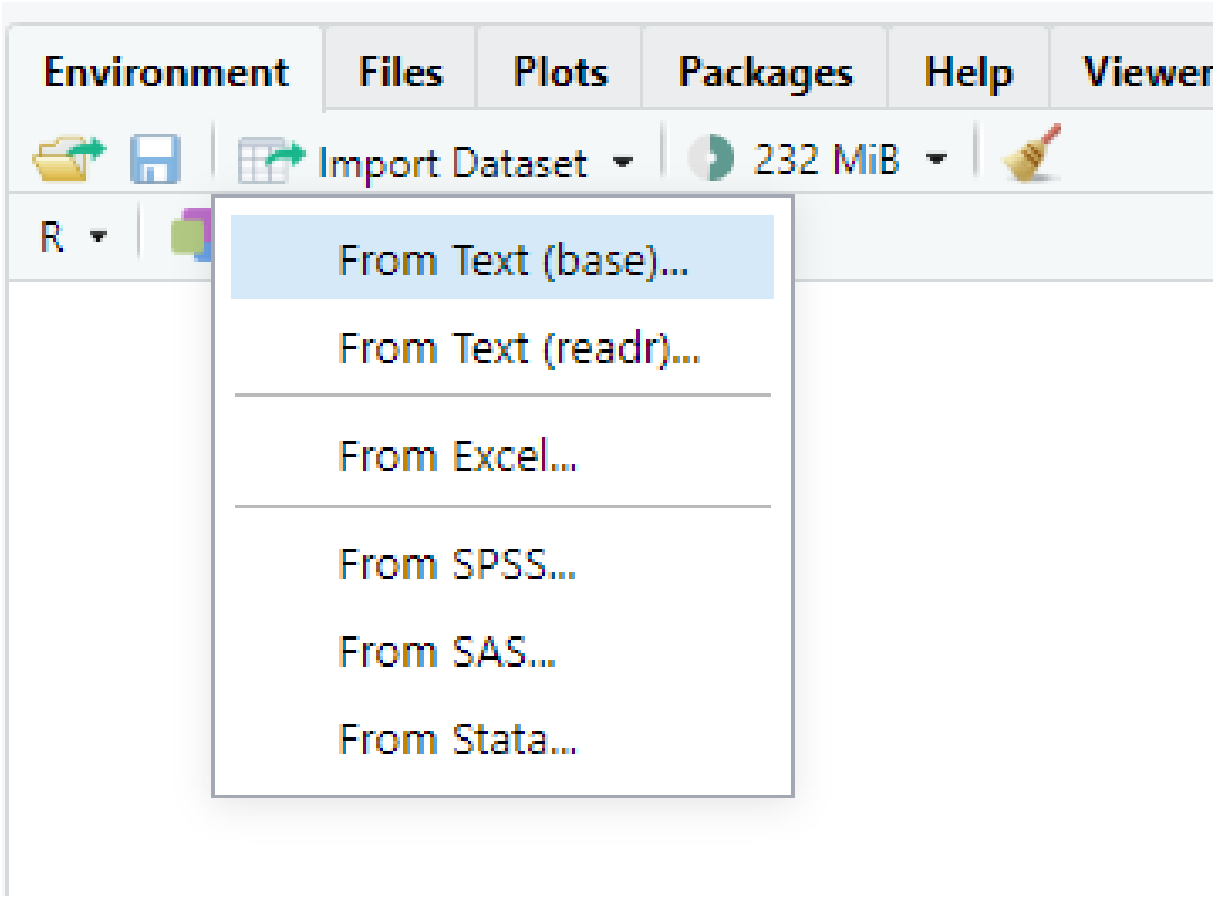
```
# Lettura file dati esterni
data <- read.csv2("data/dati.csv")
```

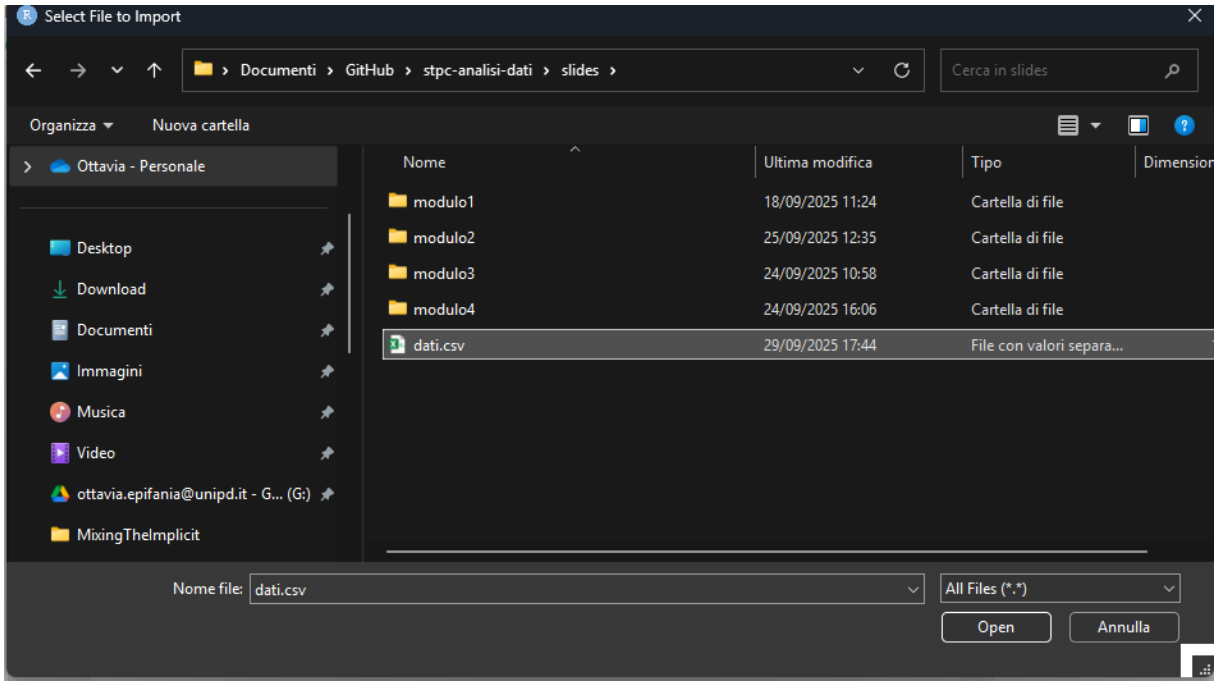
> Caution
>
> Nei dataset messi a disposizione per questo corso, si deve sempre usare la funzione `read.csv2()`.
> Perché il codice di sopra funzioni, si assume che la cartella `data` sia all'interno del progetto R creato per il corso (vedi Section **??**).

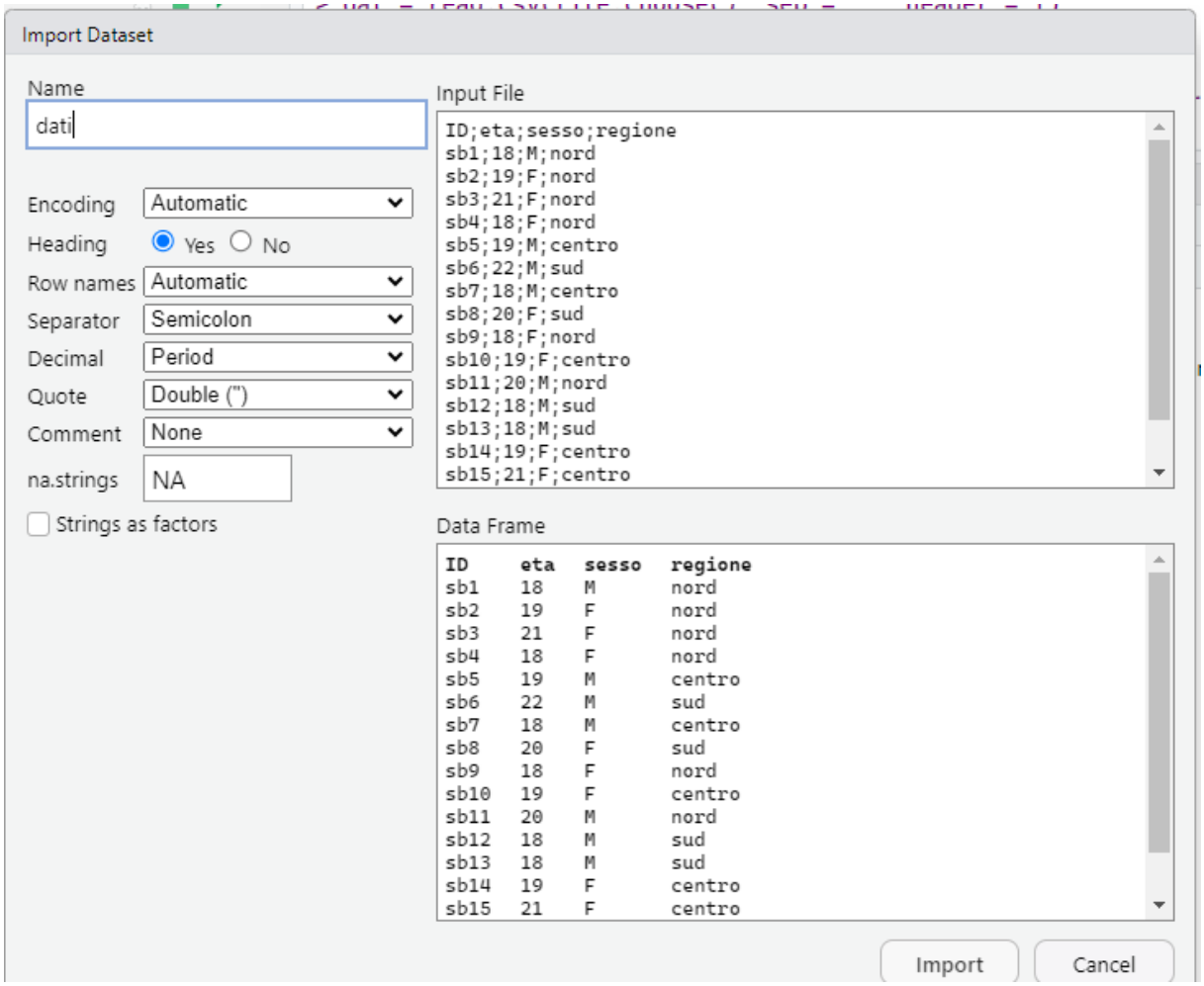
1.14.2 Importa alternativa 1

> Warning
>
> Questa alternativa è presentata solo per completezza di informazione, ma è fortemente sconsigliata visto che non può essere automatizzata dentro i documento quarto.


Environment
Files
Plots
Packages
Help
Viewer
Presentation
Import Dataset
231 MiB
R
Global Environment


Environment
Files
Plots
Packages
Help
Viewer
Import Dataset
232 MiB
R
From Text (base)...
From Text (readr)...
From Excel...
From SPSS...
From SAS...
From Stata...


Select File to Import
Documenti › GitHub › stpc-analisi-dati › slides ›
Cerca in slides
Organizza
Nuova cartella
Ottavia - Personale
Desktop
Download
Documenti
Immagini
Musica
Video
ottavia.epifania@unipd.it - G... (G:)
MixingTheImplicit
Nome
Ultima modifica
Tipo
Dimensior
modulo1
18/09/2025 11:24
Cartella di file
modulo2
25/09/2025 12:35
Cartella di file
modulo3
24/09/2025 10:58
Cartella di file
modulo4
24/09/2025 16:06
Cartella di file
dati.csv
29/09/2025 17:44
File con valori separa...
Nome file:
dati.csv
All Files (*.*)
Open
Annulla


Import Dataset
Name
dati
Encoding
Automatic
Heading
Yes
No
Row names
Automatic
Separator
Semicolon
Decimal
Period
Quote
Double (")
Comment
None
na.strings
NA
Strings as factors
Input File
ID;eta;sesso;regione
sb1;18;M;nord
sb2;19;F;nord
sb3;21;F;nord
sb4;18;F;nord
sb5;19;M;centro
sb6;22;M;sud
sb7;18;M;centro
sb8;20;F;sud
sb9;18;F;nord
sb10;19;F;centro
sb11;20;M;nord
sb12;18;M;sud
sb13;18;M;sud
sb14;19;F;centro
sb15;21;F;centro
Data Frame
ID eta sesso regione
sb1 18 M nord
sb2 19 F nord
sb3 21 F nord
sb4 18 F nord
sb5 19 M centro
sb6 22 M sud
sb7 18 M centro
sb8 20 F sud
sb9 18 F nord
sb10 19 F centro
sb11 20 M nord
sb12 18 M sud
sb13 18 M sud
sb14 19 F centro
sb15 21 F centro
Import
Cancel

1.14.3 Importa alternativa 2

> Warning
>
> Questa funzione è la funzione base su cui poggia `read.csv2()`. Cambiano i default, che in questo caso sono ottimizzati per i file csv creati in altre regioni del mondo, dove "," è il sepratore di colonna di default e "." è il separatore di decimali di default.

```
dati = read.csv("slides/dati.csv",
                sep =",", dec =".", header = TRUE)
head(dati)
```

```
  ID.eta.sesso.regione
1        sb1;18;M;nord
2        sb2;19;F;nord
3        sb3;21;F;nord
4        sb4;18;F;nord
5      sb5;19;M;centro
6         sb6;22;M;sud
```

1.15 Sintesi delle strutture tabellari

1.15.1 Riassunto descrittivo: `summary()`

```
summary(data)
```

Riassume il contenuto di strutture di dati, riportando diverse statistiche descrittive (Table **??**). Tutte le statistiche descrittive possono essere ottenute con funzioni equivalenti.

Table 1.8: Statistiche restituite dalla funzione `summary()` per una variabile numerica

Statistica	Nome	Cosa indica	Funzione equivalente
`Min.`	Minimo	Il valore più piccolo osservato	`min(x)`
`1st Qu.`	Primo quartile (Q_1)	Il valore preceduto dal 25% delle osservazioni e superato dal restante 75%	`quantile(x, 0.25)`
`Median`	Mediana (Q_2)	Il valore centrale della distribuzione ordinata: il 50% delle osservazioni sta sotto, il 50% sopra	`median(x)`
`Mean`	Media aritmetica	Il baricentro della distribuzione: la somma dei valori divisa per il numero di osservazioni	`mean(x)`
`3rd Qu.`	Terzo quartile (Q_3)	Il valore preceduto dal 75% delle osservazioni e superato dal restante 25%	`quantile(x, 0.75)`
`Max.`	Massimo	Il valore più grande osservato	`max(x)`
`NA's`	Valori mancanti	Il numero di osservazioni mancanti (compare solo se ce ne sono)	`sum(is.na(x))`

```
summary(rock)
```

```
      area            peri            shape             perm       
 Min.   : 1016   Min.   : 308.6   Min.   :0.09033   Min.   :   6.30  
 1st Qu.: 5305   1st Qu.:1414.9   1st Qu.:0.16226   1st Qu.:  76.45  
 Median : 7487   Median :2536.2   Median :0.19886   Median : 130.50  
 Mean   : 7188   Mean   :2682.2   Mean   :0.21811   Mean   : 415.45  
 3rd Qu.: 8870   3rd Qu.:3989.5   3rd Qu.:0.26267   3rd Qu.: 777.50  
 Max.   :12212   Max.   :4864.2   Max.   :0.46413   Max.   :1300.00  
```

1.15.2 Struttura: `str()`

```
str(data)
```

Fornisce informazioni circa la struttura generale di un oggetto (qualsiasi oggetto):

```
str(rock) #dataframe
```

```
'data.frame':   48 obs. of  4 variables:
 $ area : int  4990 7002 7558 7352 7943 7979 9333 8209 8393 6425 ...
 $ peri : num  2792 3893 3931 3869 3949 ...
 $ shape: num  0.0903 0.1486 0.1833 0.1171 0.1224 ...
 $ perm : num  6.3 6.3 6.3 6.3 17.1 17.1 17.1 17.1 119 119 ...
```

```
str(weight)
```

```
 num [1:6] 17.87 11.52 15.82 17.48 6.33 ...
```

1.15.3 L'inizio `head()` e la fine `tail()` di un oggetto

```
head()
```

Le prime 6 osservazioni del dataset

```
head(rock)
```

```
  area    peri     shape perm
1 4990 2791.90 0.0903296  6.3
2 7002 3892.60 0.1486220  6.3
3 7558 3930.66 0.1833120  6.3
4 7352 3869.32 0.1170630  6.3
5 7943 3948.54 0.1224170 17.1
6 7979 4010.15 0.1670450 17.1
```

```
tail()
```

le ultime 6 osservazioni del dataset

```r
tail(rock)
```

```
   area     peri    shape perm
43 5605 1145.690 0.464125 1300
44 8793 2280.490 0.420477 1300
45 3475 1174.110 0.200744  580
46 1651  597.808 0.262651  580
47 5514 1455.880 0.182453  580
48 9718 1485.580 0.200447  580
```

1.15.4 `apply()`

Ci sono diverse funzioni in R che permettono di applicare la stessa funzione più volte su uno stesso oggetto, come ad esempio la funzione `apply()`:

```r
apply(data, MARGIN, FUN)
```

`data`: i dati su cui si vuole lavorare. Si può prendere anche una sottoselezione di dati utilizzando l'indicizzazione del dataframe (sia per le righe sia per le colonne, vedi Section **??**).

`MARGIN = 1`: applica la funzione sulle righe (attraverso le colonne). Questo vuol dire che si ottiene un vettore dei risultati di lunghezza uguale al numero di righe del dataframe.

`MARGIN = 2`: applica la funzione sulle colonne (attraverso le righe). Questo vuol dire che si ottiene un vettore dei risultati di lunghezza uguale al numero di colonne del dataframe.

`FUN`: la funzione che si vuole applicare

```r
apply(rock, 1, sum)   # Somma per riga
```

```
 [1]  7788.290 10901.049 11495.143 11227.737 11908.762 12006.417 13696.040
 [8] 12571.014 12194.244  9642.812 13963.201 12729.388 14770.169 12468.672
[15] 13498.943 12585.623 15629.464 15589.483 16801.020 14405.155 11408.854
[22] 16371.431 17051.890 11893.602  9077.671  6312.529 10072.432  7571.553
[29]  9250.435  7025.001  9393.452 10245.336  8356.958  7258.715  9692.914
[36] 10305.336  4945.877  2044.761  4813.624  7012.214  7289.872  2624.872
[43]  8051.154 12373.910  5229.311  2829.071  7550.062 11783.780
```

```r
apply(rock, 2, mean)  # Media per colonna
```

```
        area         peri        shape         perm
7187.7291667 2682.2119375    0.2181104  415.4500000
```

2 Modulo 2 – Scale di misura e statistica descrittiva univariata

2.1 Tipi di variabili

QUALITATIVE / CATEGORIALI

- Sconnesse: $\{a, b, c\}$
- Ordinate: $\{a, b, c\}$ con $a < b < c$ (oppure $a > b > c$)

QUANTITATIVE / METRICHE

- Discrete: $\mathbb{Z}$
- Continue: $\mathbb{R}$

2.2 Scale di misura (Stevens, 1946)

Le variabili si differenziano in base alla quantità di informazione che può essere ricavata.

- Variabili qualitative
  - **Nominale**: Distingue un insieme di dati in diverse categorie
  - **Ordinale**: Distingue un insieme di dati in diverse categorie che sono ordinabili a seconda della quantità di caratteristica posseduta
- Variabili quantitative
  - **Intervalli equivalenti** Distingue un insieme di dati in diverse categorie ordinabili. La distanza tra le categorie è nota perché c'è un'unità di misura
  - **A rapporti equivalenti**: Come la scala ad intervalli...ma lo 0 indica assenza di caratteristica (non è arbitrario ma è assoluto)

2.2.1 Categoriali - Nominali

Comune di residenza, sesso biologico, patologia clinica ecc.

2.2.1.1 `as.factor()`

`as.factor()`: Converte una variabile in una variabile di tipo **categoriale/nominale** con tanti livelli quanti sono i valori unici nel vettore

Due livelli

```
Genere <- c("m","f","f","m","f","m")
Genere
```

```
[1] "m" "f" "f" "m" "f" "m"
```

```
as.factor(Genere)
```

```
[1] m f f m f m
Levels: f m
```

`as.character()` produre una variabili categoriale, ma senza i livelli che caratterizzano i `factor`

Più livelli

```
diagnosi <- c("schizo", "dep", "narc", "schizo", "dep", "narc")
diagnosi
```

```
[1] "schizo" "dep"    "narc"   "schizo" "dep"    "narc"
```

```
as.factor(diagnosi)
```

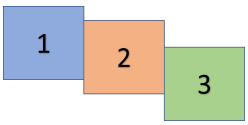
```
[1] schizo dep    narc   schizo dep    narc
Levels: dep narc schizo
```

2.2.2 Scala ordinale

Distingue un insieme di dati in diverse categorie che sono ordinabili a seconda della quantità di caratteristica posseduta. Vale sia l'equivalenza ($=$ o $\neq$) sia l'ordinamento ($<$ o $>$).

Esempi: Titolo di studio, gradimento di un prodotto, valutazione di una malattia (alto, medio, basso).

Le etichette numeriche valgono per il loro ordine (non ha senso compiere operazioni tra di loro).



> Attenzione!
>
> Si conosce l'ordine tra le categorie, non si conosce la distanza.
> Le etichette numeriche valgono per il loro ordine (non ha senso compiere operazioni tra di loro)

```
factor(A,levels=c("a1","a2",..., "ak"), ordered=TRUE)
```

Converte la variabile in una variabile con tanti livelli quanti sono i valori unici e viene forzato un ordine su questi livelli:

Listing 2.1 Utilizzo dell'agormento levels per ordinare i livelli di una variabile categoriale ordinata

```
Voto <- c("alto","medio","medio","basso","alto")
Voto
```

```
[1] "alto"  "medio" "medio" "basso" "alto"
```

Listing 2.2 Utilizzo dell'agormento levels per ordinare i livelli di una variabile categoriale ordinata

```
factor(Voto,levels=c("basso","medio","alto"))
```

```
[1] alto  medio medio basso alto
Levels: basso medio alto
```

> Warning
>
> L'utilizzo dell'argomento ordered è deprecato, sconsiglio di usarlo. In caso di variabili ordinali, è sufficiente ordinare i livelli attraverso l'argomento levels come mostrato oin Listing **??**.

2.2.3 Scala a intervalli equivalenti

Lo 0 è arbitrario, ma c'è un'unità di misura per cui le categorie si trovano alla stessa distanza. Vale equivalenza ($=$ o $\neq$), ordinamento ($<$ o $>$) e differenza ($+$ o $-$).

Esiste una relazione d'ordine tra le quantità e la distanza tra le quantità è nota e definita da una unità di misura costante e **nota**.

Esempi: Temperatura in Celsius, Q.I., scale di atteggiamento.

Se ci sono 0°C, non vuol dire che non c'è calore!

Prendiamo due temperature, T_a e T_b. Queste temperature in Celsius C sono $T_a(C) = 5$ e $T_b(C) = 10$.

Quello che si può dire:

- $T_a(C) < T_b(C)$ (oppure $T_b(C) > T_a(C)$)
- $T_b(C) - T_a(C) = 5$
- Non possiamo dire che $T_b(C) = 2T_a(C)$.

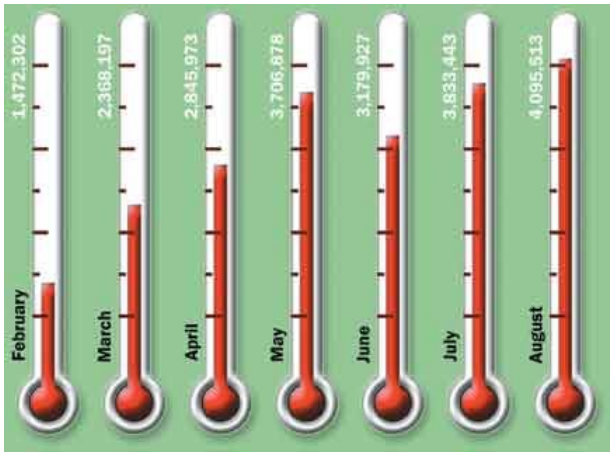
Infatti, volendo trasformare le due temperature in Fahrenheit tramite la relazione:

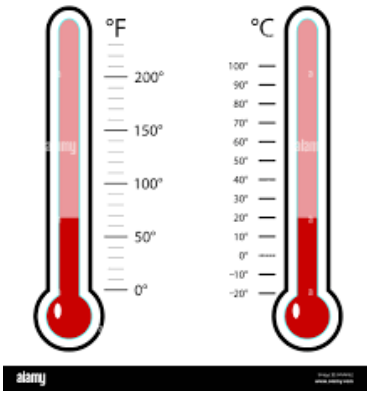
$$F = \frac{9}{5}C + 32$$

si vedrebbe che $T_a(F) = \frac{9}{5}5 + 32 = 41$ e $T_a(F) = \frac{9}{5}10 + 32 = 50$.

Rimane la relazione d'ordine, per cui anche $T_a(F) < T_b(F)$, ma cambia sia la differenza tra le due quantità $T_b(F) - T_a(F) = 9$ sia la relazione tra le due, per cui si vede chiaramente che $T_a(F) \neq T_b(F)$.

-3	-2	-1	0	1	2	3





2.2.4 Scala a rapporti equivalenti

Lo 0 è assoluto: indica assenza del fenomeno. Vale equivalenza ($=$ o $\neq$), ordinamento ($<$ o $>$), differenza ($+$ o $-$) e rapporto ($\times$ o $\div$).

Esempi: Peso, altezza, valori diagnostici.

La presenza dello 0 assoluto permette di andare oltre la mera relazione d'ordine e distanza tra le quantità.

Si prendano ad esempio due pesi, P_a e P_b. Questi pesi, misurati in chilogrammi K, sono $P_a(K) = 10$ e $P_b(K) = 30$. Quello che si può dire:

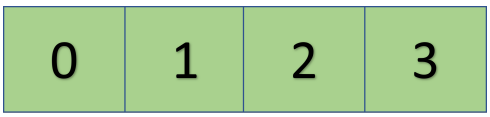
- $P_a(K) < P_b(K)$ $(P_b(K) > P_a(K))$
- $P_b(K) - P_a(K) = 20$
- Ma soprattutto $P_a(K) = \frac{1}{3}P_b(K)$ oppure $P_b(K) = 3P_a(K)$

La relazione tra P_a e P_b rimane **invariante** attraverso le diverse trasformazioni di scala.

Ad esempio, trasformando $P_a(K)$ e $P_b(K)$ in libbre (L) mediante la relazione

$$L = K \times 2.20462$$

per cui $P_a(L) = 10 \times 2.20462 = 22.0462$ e $P_b(L) = 20 \times 2.20462 = 44.0924$. La relazione d'ordine è matenuta e, al netto dei decimali, rimane anche invariante il rapporto tra i due pesi.





2.2.4.1 `numeric`, `integer()`

Le **variabili metriche** in R sono variabili di tipo `numeric` o `integer`, a seconda che siano continue o discrete

> Attenzione!
>
> In R non esiste una distinzione tra variabili a intervalli equivalenti e variabili a rapporti equivalenti – Sta a voi capire che tipo di variabile sia

```
x = seq(-3, 3, by = .5); x                    ①
y = rnorm(10, mean = 3, sd=2); y              ②
z = 1:15; z                                   ③
```

① La funzione `seq()` genera un vettore di numeri da un minimo a un massimo, con un step progressivo definito dall'utente
② La funzione `rnorm()` genera un vettore di lunghezza definita estraendo a caso da una distribuzione normale
③ L'operatore `:` genera una sequenza unitaria di valori da un minimo a un massimo

```
 [1] -3.0 -2.5 -2.0 -1.5 -1.0 -0.5  0.0  0.5  1.0  1.5  2.0  2.5  3.0
 [1]  2.59480323  2.36039338  7.12456219  1.15997385  3.84418637 11.09080089
 [7]  0.03020632  6.05966558  5.64802227  7.46831701
 [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
```

```
str(x)
```

```
 num [1:13] -3 -2.5 -2 -1.5 -1 -0.5 0 0.5 1 1.5 ...
```

```
str(y)
```

```
 num [1:10] 2.59 2.36 7.12 1.16 3.84 ...
```

```
str(z)
```

```
 int [1:15] 1 2 3 4 5 6 7 8 9 10 ...
```

2.3 Frequenze

Le frequenze sono le statistiche descrittive più semplici che si possano calcolare su una variabile: indicano quante volte compare ciascuna categoria (o ciascun valore). Si possono calcolare sia sulle variabili categoriali sia su quelle quantitative, ma sono più informative per le prime. Nelle variabili quantitative, soprattutto se continue, quasi ogni valore compare una sola volta, e le frequenze diventano utili solo dopo aver raggruppato i valori in classi.

2.3.1 Assolute e relative

Categoria x_i	n_i	$\%_i = \frac{n_i}{n} \cdot 100$
x_1	n_1	$\%_1$
x_2	n_2	$\%_2$
$\vdots$	$\vdots$	$\vdots$
x_k	n_k	$\%_k$
Totale	n	100

Frequenza assoluta: La frequenza assoluta semplice di una categoria è il **numero naturale di unità statistiche** che presentano tale categoria. $n_i \geq 0$, con $n_i \in \mathbb{N}$ e $\sum_{i=1}^{k} n_i = n$

Frequenza percentuale: La frequenza relativa semplice (o **frequenza percentuale**) è data dal rapporto tra la frequenza assoluta semplice di tale categoria e il numero totale di unità statistiche osservate. $\%_i = \frac{n_i}{n} \cdot 100$, con $0 \leq \%_i \leq 100$ e $\sum_{i=1}^{k} \%_i = 100$

2.3.1.1 Frequenze univariate: `table()`

```
table(variabile)
```

Calcola le frequenze per ogni categoria. Funziona sia per le variabili categoriali sia per le variabili metriche (anche se ha poco senso).

```
table(Genere)
```

```
Genere
f m
3 3
```

```
table(diagnosi)
```

```
diagnosi
   dep   narc schizo
     2      2      2
```

2.3.2 Cumulate

Frequenza cumulata: numero o percentuale di unità statistiche che presentano una categoria "minore o uguale" alla categoria corrente.

Frequenze assolute cumulate

$$N_i = n_1 + n_2 + \cdots + n_i = \sum_{j=1}^{i} n_j$$

con $N_1 = n_1$ e $N_k = n$.

Frequenze percentuali cumulate

$$N_i\% = n_1\% + n_2\% + \cdots + n_i\% = \sum_{j=1}^{i} n_j\%$$

con $N_1\% = n_1\%$ e $N_k\% = 100\%$.

Le frequenze cumulate possono essere definite **ricorsivamente**:

$$N_i = N_{i-1} + n_i, \qquad N_i\% = N_{i-1}\% + n_i\%$$

> Attenzione
>
> Le frequenze cumulate si calcolano **solo quando le modalità del carattere presentano un ordinamento**.
> Non hanno quindi senso per caratteri qualitativi sconnessi (nominali).

Esprimi l'accordo con l'affermazione:

> Insegnare statistica a psicologia è una pessima idea

Modalità x_i	n_i	N_i	n_i%	N_i%
Totalmente in disaccordo	10	10	24.4%	24.4%
Abbastanza in disaccordo	8	18	19.5%	43.9%
Indifferente	6	24	14.6%	58.5%
Abbastanza d'accordo	14	38	34.2%	92.7%
Totalmente d'accordo	3	41	7.3%	100.0%
Totale	41		100.0%	

2.3.2.1 Calcolare le frequenze cumulate: `cumsum()`

```
opinione = data.frame(
  categoria = c("Totalmente in disaccordo", "Abbastanza in disaccordo",
    "Indifferente",
    "Abbastanza d'accordo", "Totalmente d'accordo"),
  n = c(10, 8, 6, 14, 3)
)
opinione$N = cumsum(opinione$n)
opinione
```

```
                categoria  n  N
1 Totalmente in disaccordo 10 10
2 Abbastanza in disaccordo  8 18
3             Indifferente  6 24
4     Abbastanza d'accordo 14 38
5     Totalmente d'accordo  3 41
```

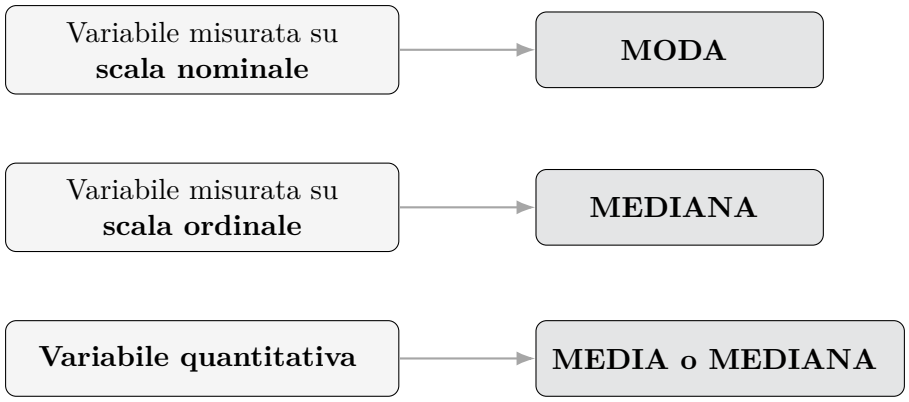
2.4 Indici di posizione

- **indici sintetici** che evidenziano le caratteristiche essenziali della distribuzione della variabile

  Presentano livelli maggiori di machiavellismo l3 CEO o l3 impiegat3?

- Si confrontano categorie che rappresentano i livelli o valori tipici.
- Consentono di sintetizzare un insieme di misure tramite un unico valore "rappresentativo" → indice che riassume o descrive i dati e dipende dalla scala di misura dei dati in oggetto.
- moda
- mediana
- quartili/percentili
- media aritmetica

Ogni variabile statistica ha l'indice di posizione adeguato: non tutti gli indici si possono calcolare per ogni variabile → **indice di posizione adeguato**



2.4.1 Moda (Mo)

La moda Mo di una variabile X è la categoria a cui è associata la massima frequenza:

$$Mo(X) = \max(n_i)$$
(categoria con massimo valore di n_i)

> **Warning**
>
> N.B: Si può calcolare per le variabili misurate a qualsiasi livello, anche se di fatto viene calcolata solo per le variabile categoriali, in quanto per altri livelli di misurazione si possono calcolare altri indici più informativi

```
opinione
```

```
                 categoria  n  N
1 Totalmente in disaccordo 10 10
2 Abbastanza in disaccordo  8 18
3             Indifferente  6 24
4     Abbastanza d'accordo 14 38
5     Totalmente d'accordo  3 41
```

```
max(opinione$n) # restituisce max(n_i)
```

```
[1] 14
```

```
# va filtrato il dataframe per ottenere la categoria a cui
# è associata la frequenza massima
opinione[opinione$n == max(opinione$n), "categoria"]
```

```
[1] "Abbastanza d'accordo"
```

```
# codice alternativo che produce lo stesso risultato
opinione$categoria[opinione$n == max(opinione$n)]
```

```
[1] "Abbastanza d'accordo"
```

2.4.2 Mediana

La mediana Me di una variabile X è la categoria che occupa la posizione centrale o mediana (Pos_{Me}) nella distribuzione ordinata dei dati, ovvero che è:

- preceduta da almeno 50% dei casi
- superata da almeno 50% dei casi

$$Pos_{Me} = \frac{(N+1)}{2}$$

Il valore di Pos_{Me} va cercato nella distribuzione delle frequenze cumulate e si sceglie la categoria con $N_i \geq Pos_{Me}$

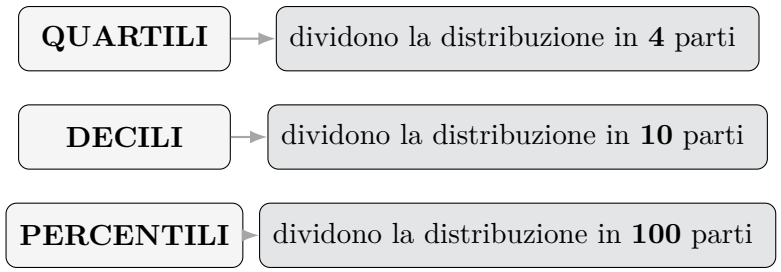
2.4.3 Quantili

Categorie/valori che dividono la distribuzione di frequenza ordinata in più parti.

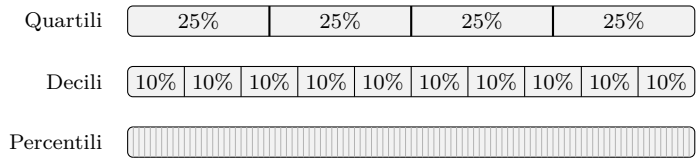
> Qual è il reddito familiare che divide il 25% dei più poveri dal restante 75% ?
>
> Qual è la soglia di reddito oltre cui sta la fascia dei più abbienti ?
>
> Quali sono le persone che hanno performato al meglio nell'ultimo semestre?

Il quantile x_p di ordine p della variabile X è quella categoria/valore che è:

- preceduta da almeno p% dei casi
- superata da almeno $(1-p)$% dei casi



La distribuzione è la stessa: cambia il modo in cui viene suddivisa



Per il calcolo dei quantili di diverso ordine, il procedimento è analogo a quello della Mediana, che di fatto è un caso particolare di questi indici. In generale la Posizione di ordine p (in centesimi) si ottiene:

$$Pos_p = \frac{n+1}{100} \cdot p$$

2.4.3.1 Trovare i quantili: `quantile()`

```
quantile(x, probs = c(0, .25, .50, .75, 1))
```

Funzione per il calcolo dei quantili da una variabile X (argomento `x`) di diverso ordine (definito dall'argomento `probs`). Di default, vengono calcolati i quartili, compresi anche il minimo (`0`) e il massimo (`1`) della variabile X.

```
set.seed(123) # garantisce la replicabilità dei risultati
x = rnorm(1000, mean = 0, sd = 1)
quantile(x)
```

```
          0%          25%          50%          75%         100%
-2.809774679 -0.628324243  0.009209639  0.664601867  3.241039935
```

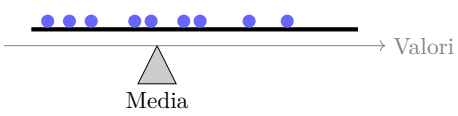
```
# calcola i decili
quantile(x, probs = seq(0,1, by = .10))
```

```
          0%          10%          20%          30%          40%          50%
-2.809774679 -1.267328289 -0.788690220 -0.486452454 -0.214854912  0.009209639
          60%          70%          80%          90%         100%
 0.242040647  0.513088142  0.841412710  1.254751947  3.241039935
```

2.4.4 Media

La media è una sorta di baricentro della distribuzione dei dati. Data una variabile X composta da n valori:

$$\bar{X} = \frac{\sum_{i=1}^{n} x_i}{n}$$



I dati: $X = (0.8, 1.2, 1.6, 2.4, 2.7, 3.3, 3.6, 4.5, 5.2)$, la media $\bar{X} = 2.811$



I dati: $X = (1.18, 1.02, 1.10, 1.16, 1.01, 1.10, 1.05, 1.12, 1.45, 1.52, 1.54, 1.55, 10.06, 14.46)$, la media $\bar{X} = 2.809$

2.4.4.1 `mean()`

```
mean(x, na.rm = FALSE)
```

Funzione per calcolare la media di un vettore di valori `x`. L'agromento `na.rm = TRUE` serve per ottenere il calcolo della media nonostante la presenza di valori NA

`rowMeans()`: Calcola la media per ogni riga (quindi attraverso le colonne)

`colMeans()`: Calcola la media per ogni colonna (quindi attraverso le righe)

```r
mean(x)
```

```
[1] 0.01612787
```

2.5 Indici di variabilità

Il concetto di variabilità si riferisce a quanto i punteggi di una distribuzione sono sparsi, ovvero quanto siano simili o dissimili tra loro.

- Il ricorso a un indice di tendenza centrale (come la media) comporta una forte semplificazione
- È fondamentale capire quanto i dati siano dispersi intorno all'indice di tendenza centrale
- La variabilità è una caratteristica fondamentale delle distribuzioni: *"Variability is the essence of statistics"* (Cobb, 1992)

Gli indici di variabilità possono assumere solo valori positivi (non ha senso parlare di dispersione negativa) o nulli (quando i dati hanno tutti lo stesso valore).

2.5.1 Campo di variazione (Gamma) o range

Il campo di variazione è la differenza tra il valore massimo e il valore minimo osservato:

$$\text{gamma} = X_{max} - X_{min}$$

```r
team_A <- c(38, 42, 44, 36, 39, 42, 38, 41)
max(team_A) - min(team_A)
```

```
[1] 8
```

> Limitazione
>
> Dipende solo da due valori (massimo e minimo) e non considera la distribuzione dei dati intermedi.

2.5.2 La varianza

La varianza σ^2 è la media degli scarti al quadrato tra i dati e la media:

$$\sigma^2 = \frac{\sum_i^n (X_i - \bar{X})^2}{n}$$

La varianza assume valore minimo 0 quando tutti i dati sono uguali tra loro e aumenta all'aumentare della dispersione.

> Important
>
> La funzione `var()` di R calcola la **varianza campionaria**, che divide per $n-1$ anziché n. Per dati descrittivi (popolazione osservata) si usa n; per stimare la varianza della popolazione da un campione si usa $n-1$.

2.5.3 La deviazione standard

La deviazione standard è la radice della varianza:

$$\sigma = \sqrt{\sigma^2}$$

Riporta l'indice di variabilità sulla **scala della variabile**.

Esempio: Se in un campione di dipendenti la variabile `WorkHoursPerWeek` ha media 40 ore e deviazione standard 2.5, i dipendenti differiscono mediamente di 2.5 ore dal carico di lavoro medio.

La deviazione standard, a differenza della varianza, è espressa nella stessa unità di misura della variabile.

2.5.4 Il coefficiente di variazione

Il coefficiente di variazione è il rapporto tra la deviazione standard e il valore assoluto della media:

$$CV = \frac{\sigma}{|\bar{X}|}$$

Il CV è un indice di **variabilità relativa** fondamentale per eseguire **confronti** tra fenomeni misurati su **scale diverse**.

Table 2.1: Dataset di Anscombe

Dataset	Media.X	Media.Y	Varianza.X	Varianza.Y	Correlazione
Dataset 1	9	7.5	11	4.13	0.82
Dataset 2	9	7.5	11	4.13	0.82
Dataset 3	9	7.5	11	4.12	0.82
Dataset 4	9	7.5	11	4.12	0.82

2.5.5 Differenza interquartilica (IQR)

La differenza interquartilica è la differenza tra il terzo e il primo quartile:

$$IQR = Q_{75} - Q_{25}$$

L'IQR misura la dispersione del 50% centrale dei dati ed è un indice **robusto** perché i valori estremi incidono poco o per nulla.

Outlier: valori oltre $[Q_1 - 1.5 \times IQR\ ;\ Q_3 + 1.5 \times IQR]$.

Il **boxplot** (vedi sezione Section **??**) visualizza Q1, mediana, Q3 e gli outlier in un unico grafico.

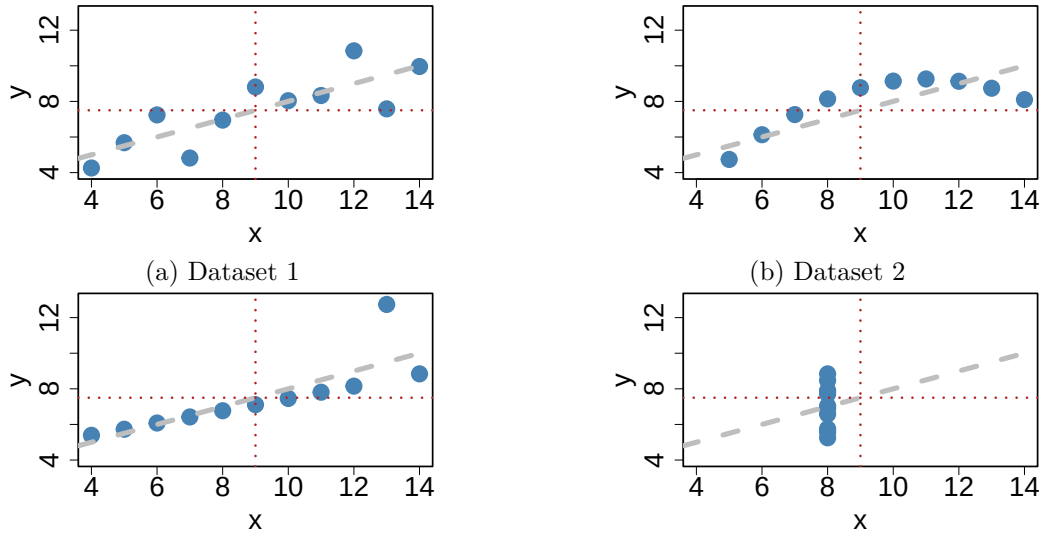
La funzione `IQR(x)` permette di calcolare la differenza interquartilica in R per qualsiasi variabile quantitativa.

2.6 Perché i grafici

> Graphs remind us that the process of statistical inference is not mechanical. This process often involves subjective decisions (e.g., the evaluation, exploration, and/or deletion of outliers) which are an integral part of the analysis. Thus, graphs are among the most appropriate tools to enhance transparency and confer plausibility to such decisions.
>
> Pastore et al. (2017)

Data set di Anscombe (Table **??**): Quattro dataset, di seguito le statistiche descrittive realitive alla variabile x e alla variabile y e la loro correlazione:

Guardando la **?@tbl-Anscombe**, sembra che i quattro dataset siano identici, in termini di media, varianza e correlazione tra le variabili x e y. Tuttavia, se si guardano i grafici a dispersione (vedi **?@sec-correlazione**) tra le variabili x e y dei quattro dataset (Figure **??**), appaiono delle differenze abbastanza palesi tra i diversi dataset.



(a) Dataset 1

(b) Dataset 2

(c) Dataset 3

(d) Dataset 4

Figure 2.1: I grafici raccontano un'altra storia

2.7 Grafici

R fornisce uno strumento molto utile per la rappresentazione grafica dei dati e dei risultati già con il suo pacchetto base `graphics`

> NB
>
> Non avete bisogno di installarlo, è caricato automaticamente nel momento in cui scaricate R e le sue funzioni sono sempre disponibili

`ggplot2` è un pacchetto più avanzato per la creazione dei grafici. Non entreremo nel dettaglio del suo funzionamento perché è un po' più complicato

Alcune funzioni incluse nel pacchetto `graphics`:

```
plot(x)                ①
plot(x, y)             ②
hist(x)                ③
boxplot(x)             ④
barplot(x)             ⑤
```

① Grafico di una variabile quantitativa
② Grafico a dispersione di due variabili quantitative (vedi **?@sec-correlazione**)
③ Istogramma di una variabile quantitativa (Section **??**)
④ Boxplot della distribuzione quartile di una (o più) variabili (Section **??**)
⑤ Grafico a barre per le frequenze delle variabili qualitative (Section **??**)

2.7.1 Grafico a barre

Usato per variabili qualitative nominali o ordinali:

- Asse x: categoria k
- Asse y: Frequenza n_k (vale anche per le frequenze percentuali)

Table 2.2: Tabelle di frequenza per le categorie A, B, C.

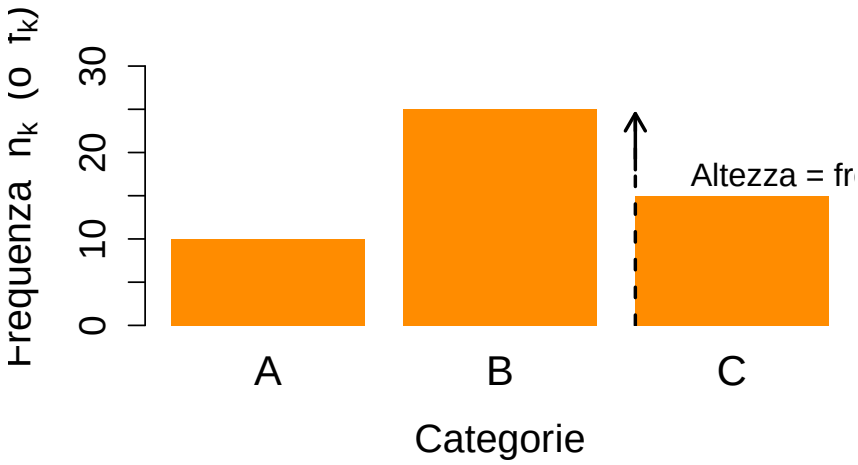
Categoria	n_k
A	21
B	24
C	22

Categoria	n_k
A	12
B	35
C	18

L'altezza della barretta è proporzionale alla sua frequenza

```
barplot(x)
```

Dove `x` è il vettore delle frequenze (quello che si ottiene applicando la funzione `table()` a una variabile categoriale)



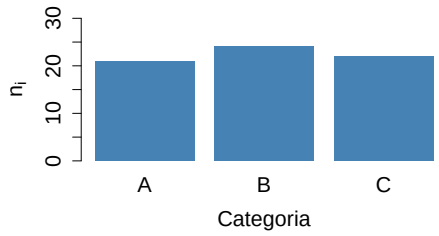


Figure 2.2: Barplot delle tabelle

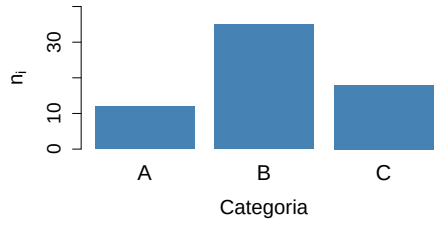


Figure 2.3: Barplot delle tabelle

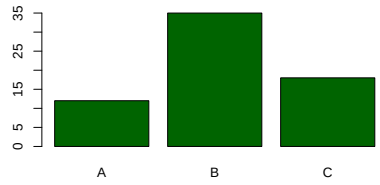
```
dati = factor(rep(c("A", "B", "C"), c(12, 35, 18))); dati
```

```
 [1] A A A A A A A A A A A A B B B B B B B B B B B B B B B B B B B B B B B B B
[39] B B B B B B B B B C C C C C C C C C C C C C C C C C C
Levels: A B C
```

```
table(dati)
```

```
dati
 A  B  C
12 35 18
```

```r
barplot(table(dati), col = "darkgreen")
```



2.7.2 Istogrammi

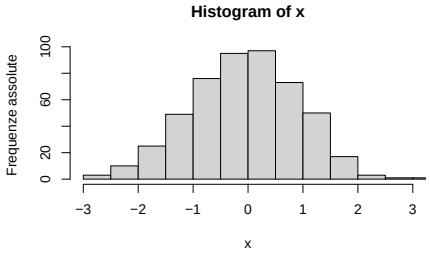
`hist(x, breaks)` produce un istogramma a partire da un vettore `x` di osservazioni quantitative.

Per costruire l'istogramma, l'intervallo dei valori di `x` viene diviso in classi contigue, disposte sull'asse delle x. Per ogni classe si conta quante osservazioni vi ricadono: questa frequenza è l'altezza della barra sull'asse delle y.

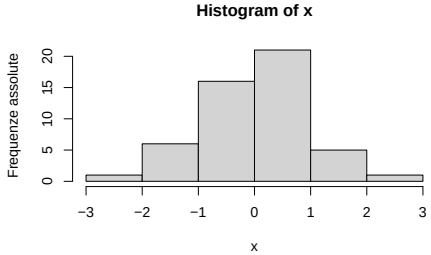
L'argomento `breaks` stabilisce come vengono costruite le classi. Può essere il numero di classi desiderato, una regola per calcolarlo (`"Sturges"`, il default, `"FD"` o `"Scott"`), oppure un vettore con i punti di taglio esatti. Il numero di classi determina la loro ampiezza:

- **Classi strette (molti `breaks`)**: ogni classe raccoglie poche osservazioni, quindi le barre sono basse e alcune classi possono restare vuote. L'istogramma risulta frastagliato e difficile da leggere.
- **Classi larghe (pochi `breaks`)**: ogni classe raccoglie molte osservazioni, quindi le barre sono più alte ma sono poche. L'istogramma diventa troppo grossolano e nasconde la forma della distribuzione.

> **Note**
>
> Quando `breaks` è un numero, R lo usa come *suggerimento* e lo arrotonda a intervalli "puliti". Per questo con `breaks = 145` non si ottengono esattamente 145 barre.



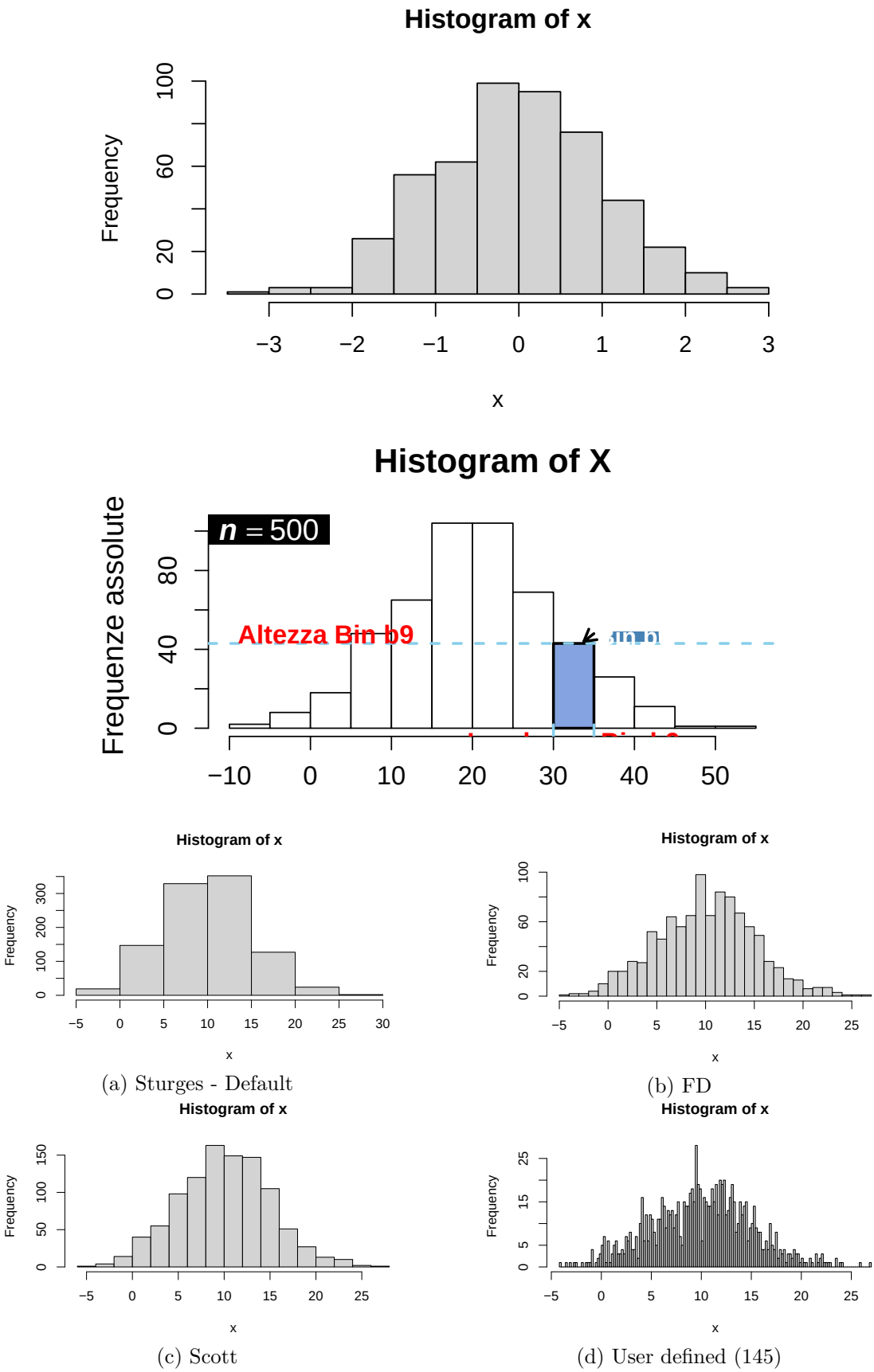
(a) n = 500



(a) n = 50

Istogrammi con n diverso

```r
x = rnorm(500, mean = 0, sd = 1)
hist(x)
```



(a) Sturges - Default

(b) FD

(c) Scott

(d) User defined (145)

Figure 2.6: Metodi per la definizione dei breaks

Sturges:

$$\frac{\max(x) - \min(x)}{\log_2(n+1)}$$

Scott:

$$3.5\sqrt{\text{var}(x)}\, n^{-1/3}$$

Freedman-Diaconis

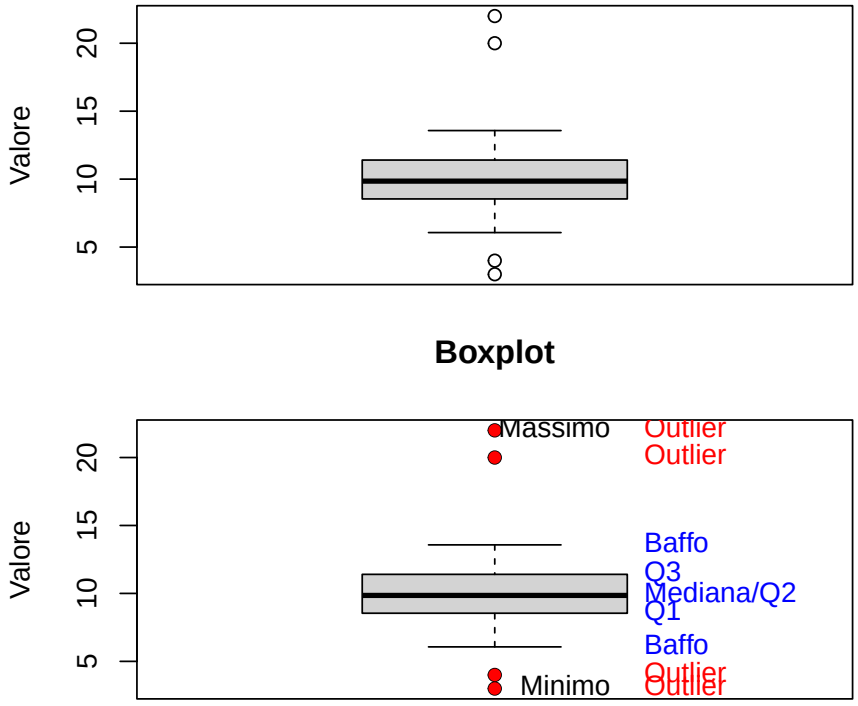
$$2\,(Q_3 - Q_1)\, n^{-1/3}$$

User defined:

$$\frac{\max(x) - \min(x)}{\text{breaks}}$$

2.7.3 Boxplot

`boxplot(x)` Produce un boxplot a partire da un vettore X di osservazioni quantitative.



2.7.4 Boxplot – Calcolo degli outlier

La lunghezza dei baffi (e di conseguenza la definizione degli outliers) è definita attraverso il metodo Tukey:

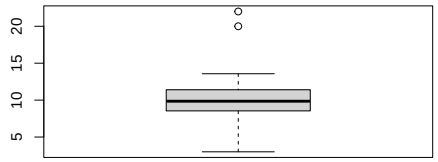
$$[Q_1 - 1.5 \times IQR,\, Q_3 + 1.5 \times IQR]$$

dove $IQR = Q_3 - Q_1$ (Inter-quartile range)

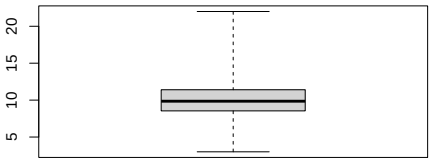
Tutti i valori che cadono al di fuori dei baffi, sono definiti outliers.

Settando l'argomento `range = x` si può cambiare il modo in cui vengono calcolati i baffi:

```r
boxplot(x, range = 2)
boxplot(x, range = 0)
```



(a) range = 2 - Meno Conservativo



(a) range = 0 - Minimo e Massimo

Calcolo degli outlier

2.8 Esempi

2.8.1 `barplot()`

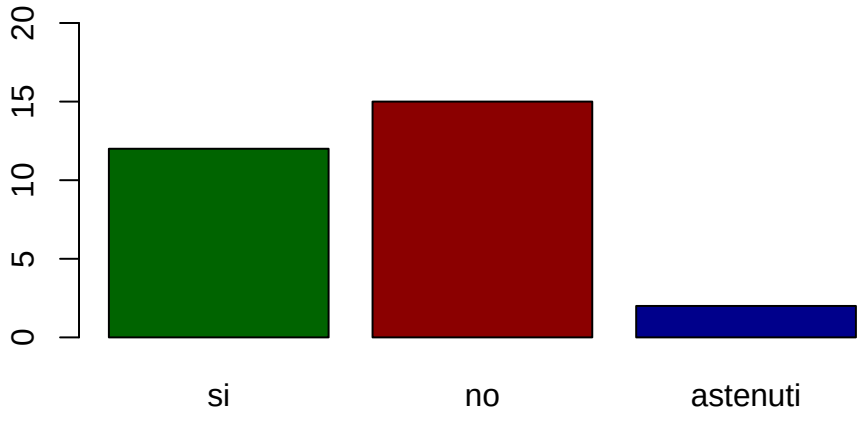
```r
voto = sample(rep(c("si", "no", "astenuti"),
           c(12, 15, 2)))
voto = factor(voto, levels = c("si", "no", "astenuti"))
table(voto)

# le frequenze restituite dalla funzione table() vanno usate
# come vettore di base per il barplot

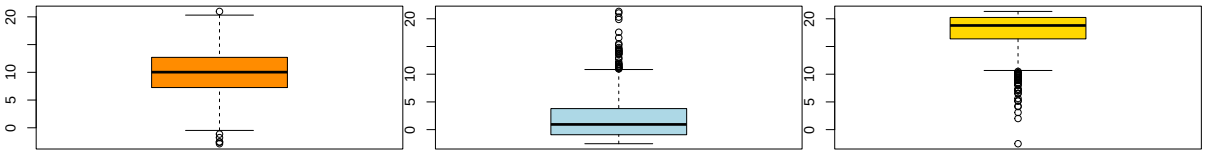
barplot(table(voto),
        col = c("darkgreen", "darkred", "darkblue"),
        ylim = c(0, 20))
```

```
voto
      si       no astenuti
      12       15        2
```

```r
barplot(table(voto),
        col = c("darkgreen", "darkred", "darkblue"),
        ylim = c(0, 20))
```



2.8.2 `boxplot()`



(a) boxplot(x) (b) boxplot(y) (c) boxplot(z)

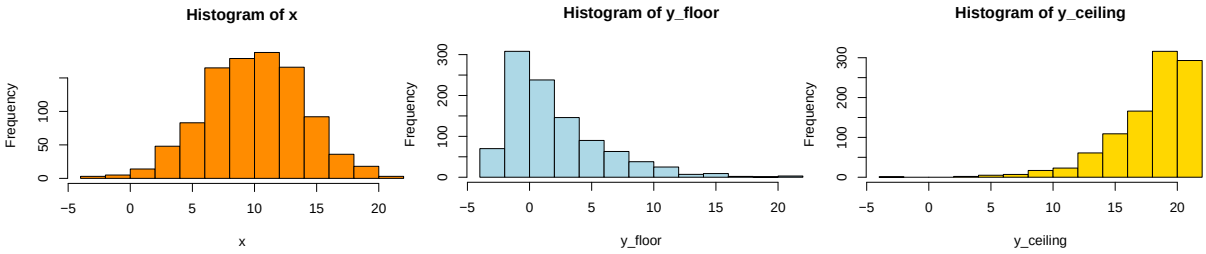
Figure 2.9: Distribuzioni a confronto

Figure **??**: La distribuzione è equilibrata attorno alla mediana, situaata circa a metà della distribzuone tra il minimo e il massimo

Figure **??**: La distribuzione è squilibrata verso il basso, ovvero verso il minimo, presentando una coda più "pesante" verso il massimo

Figure **??**: La distribuzione è squilibrata verso l'alto, ovvero verso il massimo, presentando una coda più "pesante" verso il minimo

2.8.3 `hist()`



(a) hist(x) (b) hist(y) (c) hist(z)

Figure 2.10: Distribuzioni a confronto - Le stesse dei boxplot

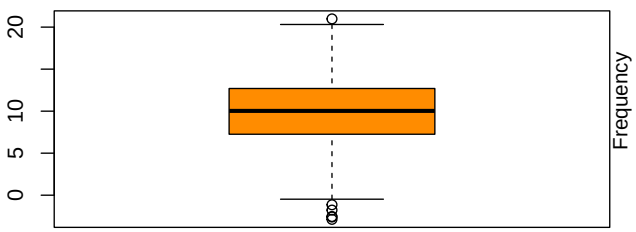
2.8.4 Confronto boxplot vs istogramma

Forzano i boxpolot ad essere in orizzontale attraverso l'argomento `horizontal = TRUE` (**?@figmyFigure**) si può apprezzare meglio la similutudine tra la forma della distribuzione vista con l'istogramma e la forma della distribuzione vista con il boxplot.

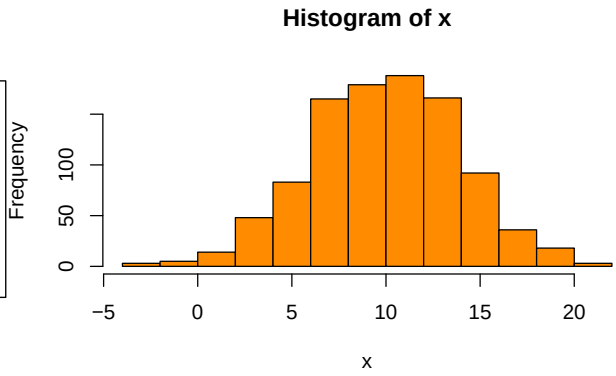
Figure **??** illustra gli stessi boxplot disposti nella stessa finestra (Listing **??**).

Listing 2.3 Finestra grafica con molti boxplot

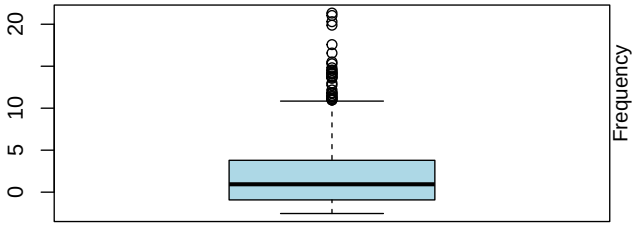
```
boxplot(x, y, z,
        names = c("Normale", "Pavimento", "Soffitto"),
        col = c("darkorange", "lightblue", "gold"))
```



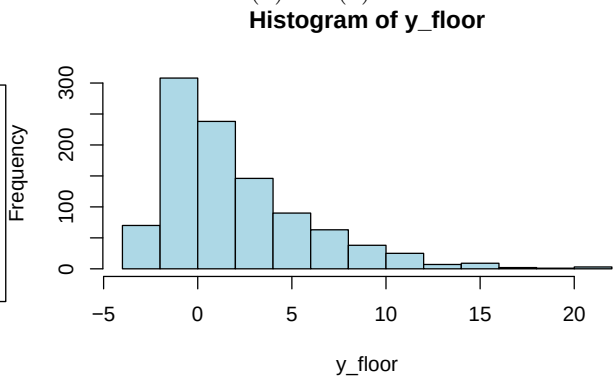
(a) boxplot(x)



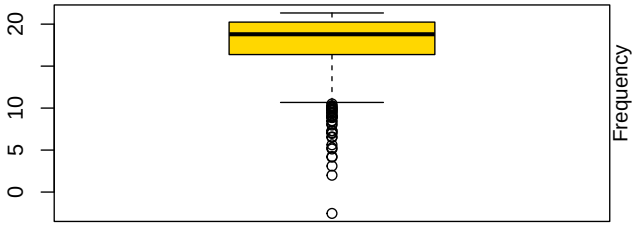
(b) hist(x)



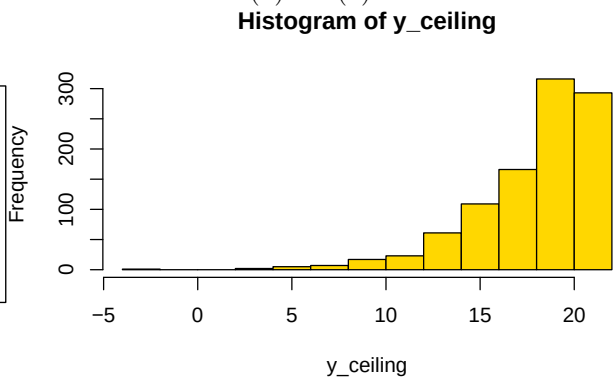
(c) boxplot(x)



(d) hist(x)

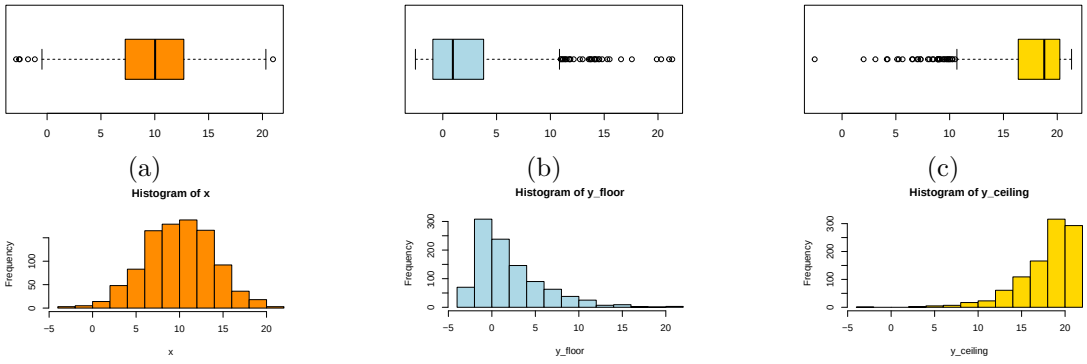


(e) boxplot(x)



(f) hist(x)

Figure 2.11: Boxplot vs Istogramma



(a) (b) (c)

(d) (e) (f)

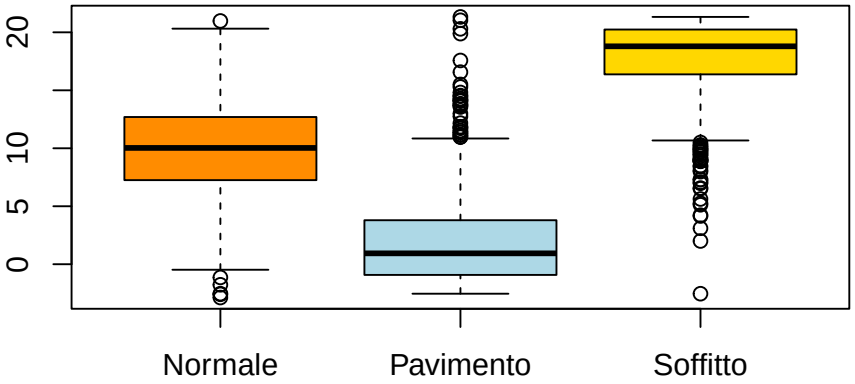


Figure 2.13: Tanti boxplot in un unico grafico

3 Modulo 3 – Statistica descrittiva bivariata

3.1 Variabile statistica doppia

Si consideri un gruppo di 11 persone, per ciascuna delle quali sono state rilevate due variabili categoriali: il titolo di studio (X, con categorie Elementare, Secondaria Inferiore e Secondaria Superiore) e l'occupazione (Y, con categorie Disoccupato, Impiegato e Dirigente). I dati sono riportati in Table **??**.

Ciascuna delle due variabili può essere descritta separatamente attraverso la sua distribuzione di frequenza. Table **??** riporta le frequenze del titolo di studio ($n_{k.}$, il numero di persone che possiedono la categoria x_k), mentre Table **??** riporta quelle dell'occupazione ($n_{.j}$, il numero di persone che possiedono la categoria y_j).

Le distribuzioni univariate non dicono nulla su come le due variabili si combinano tra loro: sappiamo quante persone hanno un titolo di studio elementare e quante sono dirigenti, ma non quante persone con un titolo elementare sono dirigenti. Per rispondere a questa domanda bisogna considerare le due variabili *congiuntamente*, contando quante persone presentano ciascuna coppia di categorie (x_k, y_j). Il risultato è la **tabella a doppia entrata** (o tavola di contingenza) in Table **??**: le righe corrispondono alle categorie di X, le colonne alle categorie di Y, e ogni cella contiene la **frequenza congiunta** n_{kj}, cioè il numero di persone che possiedono contemporaneamente la categoria x_k e la categoria y_j.

Ad esempio, la cella all'incrocio tra la riga SS e la colonna D indica che 3 persone hanno un titolo di studio di secondaria superiore *e* sono dirigenti.

Sommando le frequenze congiunte lungo ciascuna riga si ottengono i totali di riga (in rosso), mentre sommandole lungo ciascuna colonna si ottengono i totali di colonna (in blu). Questi totali sono detti **frequenze marginali**, perché si trovano ai margini della tabella. Confrontando Table **??** con Table **??** e Table **??** si nota che le frequenze marginali di riga coincidono esattamente con la distribuzione di frequenza di X e le frequenze marginali di colonna coincidono con la distribuzione di frequenza di Y. In altre parole, la tabella a doppia entrata contiene al suo interno anche le due distribuzioni univariate. Infine, la cella in basso a destra (11) riporta il numero totale di unità statistiche n, che si ottiene sia sommando i totali di riga sia sommando i totali di colonna.

Più in generale, date due variabili X e Y, la loro distribuzione di frequenze congiunte n_{kj} descrive quante unità statistiche presentano ciascuna coppia di categorie (x_k, y_j).

Table 3.1: Titolo di studio e occupazione rilevati su 11 persone

X: Istruzione	E	E	E	SI	SI	SI	SS	SS	SS	SS	SS
Y: Occupazione	DI	I	I	DI	I	I	D	I	I	D	D

Table 3.2: Distribuzioni di frequenza univariate.

(a) Variabile Istruzione

categoria x_k	$n_{k.}$
Elementare (E)	3
Secondaria Inferiore (SI)	3
Secondaria Superiore (SS)	5

(b) Variabile Occupazione

categoria y_j	$n_{.j}$
Disoccupato (DI)	2
Impiegato (I)	6
Dirigente (D)	3

Table 3.3: Tabella a doppia entrata di Istruzione e Occupazione

	DI	I	D	
E	1	2	0	3
SI	1	2	0	3
SS	0	2	3	5
	2	6	3	**11**

Tip

L'insieme (X, Y) delle terne

$$(x_k, y_j, n_{kj}) \qquad (k = 1, \dots, h;\ j = 1, \dots, m)$$

è detto **variabile statistica doppia**.

3.1.1 Tabella a doppia entrata – Tavola di contingenza

		Y						
		y_1	y_2	...	y_j	...	y_m	$n_{k.}$
X	x_1	n_{11}	n_{12}	...	n_{1j}	...	n_{1m}	$n_{1.}$
	x_2	n_{21}	n_{22}	...	n_{2j}	...	n_{2m}	$n_{2.}$
	$\vdots$	$\vdots$	$\vdots$		$\vdots$		$\vdots$	$\vdots$
	x_k	n_{k1}	n_{k2}	...	n_{kj}	...	n_{km}	$n_{k.}$
	$\vdots$	$\vdots$	$\vdots$		$\vdots$		$\vdots$	$\vdots$
	x_h	n_{h1}	n_{h2}	...	n_{hj}	...	n_{hm}	$n_{h.}$
		$n_{.1}$	$n_{.2}$	...	$n_{.j}$	...	$n_{.m}$	n

n_{kj}: **Frequenza congiunta**, definisce il numero di unità statistiche che possiedono congiuntamente la categoria x_k della variabile X e la categoria y_j della variabile Y

$n_{k.}$: **Frequenza marginale di** X, definisce il numero di unità statistiche che possiedono la categoria x_k della variabile X

$n_{.j}$: **Frequenza marginale di** Y, definisce il numero di unità statistiche che possiedono la categoria y_j della variabile Y

n: Numero totale di unità statistiche

$n_{k.} = \sum_{j=1}^{m} n_{kj}$ ⟶ Numero di unità statistiche che possiedono la categoria x_k della variabile X (**totale della riga** k)

$n_{.j} = \sum_{k=1}^{h} n_{kj}$ ⟶ Numero di unità statistiche che possiedono la categoria y_j della variabile Y (**totale della colonna** j)

$$\sum_{k=1}^{h}\sum_{j=1}^{m} n_{kj} = \sum_{k=1}^{h} n_{k.} = \sum_{j=1}^{m} n_{.j} = n$$

3.1.2 `table()`

`table(var1, var2)`: Produce la tabella di frequenza tra due variabili categoriali `var1` e `var2`

```
var1 <- as.factor(c("a1","a2","a1","a2","a1","a2","a1"))
var2 <- as.factor(c("b1","b1","b1","b2","b1","b2","b2"))
mytab = table(var1, var2); mytab
```

```
    var2
var1 b1 b2
  a1  3  1
  a2  1  2
```

```
sum(mytab) # permette di ottenere l'ampiezza campionaria
```

```
[1] 7
```

3.1.3 `margin.table()`

`margin.table(table, margin)`: Frequenze marginali per la tabella a doppia entrata

> Tip
>
> `margin = 1`: Frequenze marginali calcolate **attraverso le colonne**
> `margin = 2`: Frequenze marginali calcolate **attraverso le righe**

```
mytab
```

```
    var2
var1 b1 b2
  a1  3  1
  a2  1  2
```

```r
margin.table(mytab,1)
```

```
var1
a1 a2
 4  3
```

```r
margin.table(mytab,2)
```

```
var2
b1 b2
 4  3
```

3.1.4 `prop.table()`

`prop.table(T, margin = NULL)`: Probabilità condizionate:

> **Tip**
>
> `margin = 1`: Probabilità calcolate sul totale n, $p_{kj} = \frac{n_{kj}}{n}$
>
> `margin = 1`: Probabilità condizionate di riga (**attraverso le colonne**), $p_{kj} = \frac{n_{kj}}{n_{k.}}$
>
> `margin = 2`: Probabilità condizionate di colonna (**attraverso le righe**), $p_{kj} = \frac{n_{kj}}{n_{.j}}$

```r
mytab
```

```
    var2
var1 b1 b2
  a1  3  1
  a2  1  2
```

```r
prop.table(mytab)
```

```
    var2
var1        b1        b2
  a1 0.4285714 0.1428571
  a2 0.1428571 0.2857143
```

```r
prop.table(mytab,1)  # P(B|A)
```

```
    var2
var1        b1        b2
  a1 0.7500000 0.2500000
  a2 0.3333333 0.6666667
```

```r
prop.table(mytab,2)  # P(A|B)
```

```
    var2
var1        b1        b2
  a1 0.7500000 0.3333333
  a2 0.2500000 0.6666667
```

3.1.5 Frequenze percentuali e frequenze condizionate

Si consideri un campione di 140 donne per le quali sono state rilevate due variabili categoriali: l'occupazione (casalinga o lavoratrice) e il livello di istruzione (basso, medio o alto). La tabella a doppia entrata con le frequenze assolute e le marginali è riportata in Table **??**.

Table 3.4: Tabella a doppia entrata di Occupazione e Istruzione (frequenze assolute)

	Basso	Medio	Alto	Totale
Casalinghe	40	25	20	85
Lavoratrici	10	15	30	55
Totale	50	40	50	140

Guardando le frequenze assolute si potrebbe essere tentati di concludere, ad esempio, che tra le donne con istruzione alta ci sono più lavoratrici (30) che casalinghe (20), e che tra quelle con istruzione bassa ci sono molte più casalinghe (40) che lavoratrici (10). Le frequenze assolute, però, non permettono confronti corretti, perché i gruppi hanno dimensioni diverse: nel campione ci sono 85 casalinghe e solo 55 lavoratrici, e anche i tre livelli di istruzione non hanno la stessa numerosità (50, 40 e 50). Un numero più alto in una cella può dipendere semplicemente dal fatto che quel gruppo è più numeroso.

Per confrontare gruppi di dimensione diversa bisogna passare alle **frequenze percentuali condizionate**: invece di dividere ogni cella per il totale generale n (quelo che fa la funzione `prop.table(tabella)` quando non viene specificato un margin), la si divide per il totale della sua riga o della sua colonna. In questo modo ogni riga (o ogni colonna) diventa una distribuzione percentuale che somma a 100, e i gruppi diventano confrontabili.

La stessa tabella si può leggere in due modi diversi, a seconda di quale variabile si sceglie come *condizionante*, cioè di quale variabile si fissano le categorie per guardare come si distribuisce l'altra.

Un primo colpo d'occhio sulla struttura della tabella è dato dal mosaic plot in Figure **??**. La larghezza delle due colonne è proporzionale al numero di casalinghe e di lavoratrici, mentre l'altezza dei rettangoli all'interno di ciascuna colonna rappresenta la percentuale di donne con istruzione bassa, media o alta *in quel gruppo*.

3.1.5.1 Frequenze percentuali di riga: $P($**Istruzione** $|$ **Occupazione**$)$

Le percentuali di **riga** si ottengono dividendo ogni cella per il totale della propria riga (`prop.table(tabella, 1)`). Poiché le righe corrispondono all'occupazione, ogni riga descrive

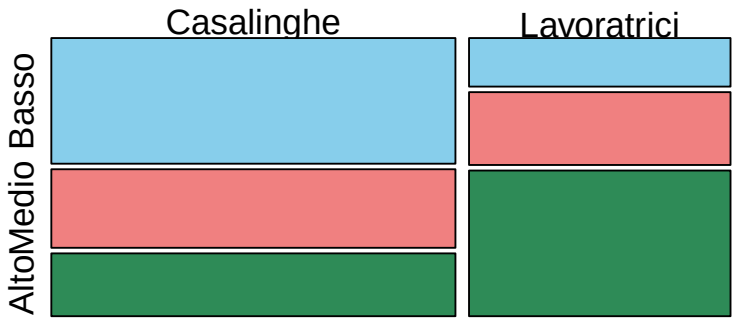


Figure 3.1: Mosaic plot di Occupazione e Istruzione

come si distribuisce il titolo di studio all'interno di ciascun gruppo occupazionale (Table **??** e Figure **??**).

Table 3.5: Frequenze percentuali di riga: distribuzione dell'Istruzione condizionata all'Occupazione

	Basso	Medio	Alto	Totale
Casalinghe	47	29	24	100
Lavoratrici	18	27	55	100

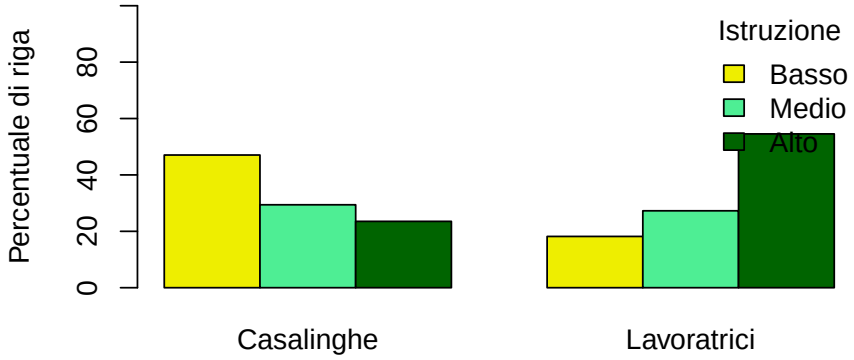


Figure 3.2: Distribuzione percentuale dell'Istruzione nei due gruppi occupazionali

La lettura corretta è sempre del tipo *"tra le donne [categoria di occupazione], il X% ha un titolo di studio [categoria di istruzione]"*. Ad esempio:

- tra le casalinghe, il 47% ha un titolo di studio basso e solo il 24% un titolo alto;
- tra le lavoratrici, il 55% ha un titolo di studio alto e solo il 18% un titolo basso.

Le percentuali di riga descrivono quindi la **composizione** di ciascun gruppo occupazionale: rispondono a domande del tipo *"che titolo di studio hanno le casalinghe?"*.

3.1.5.2 Frequenze percentuali di colonna: $P($**Occupazione** | **Istruzione**$)$

Le percentuali di **colonna** si ottengono dividendo ogni cella per il totale della propria colonna (`prop.table(tabella, 2)`). Poiché le colonne corrispondono al titolo di studio, ogni colonna descrive *come si distribuisce l'occupazione all'interno di ciascun livello di istruzione* (Table **??** e Figure **??**).

Table 3.6: Frequenze percentuali di colonna: distribuzione dell'Occupazione condizionata all'Istruzione

	Basso	Medio	Alto
Casalinghe	80	62	40
Lavoratrici	20	38	60
Totale	100	100	100

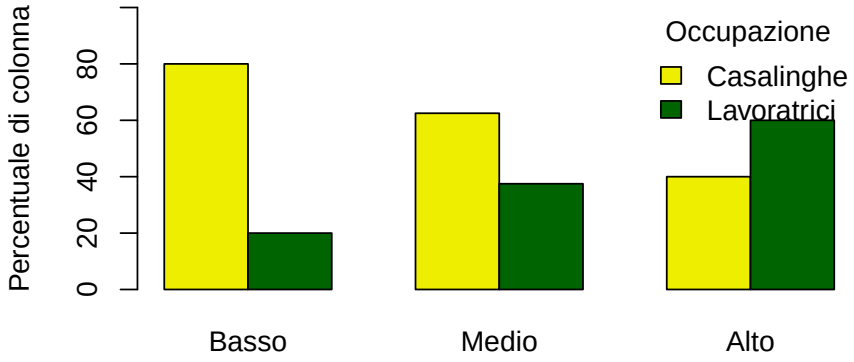


Figure 3.3: Distribuzione percentuale dell'Occupazione nei tre livelli di istruzione

- tra le donne con titolo di studio basso, l'80% è casalinga e solo il 20% lavora;
- tra le donne con titolo di studio alto, la situazione si capovolge: il 60% lavora e il 40% è casalinga.

Al crescere del titolo di studio, quindi, cresce la percentuale di lavoratrici (20%, 38%, 60%).

3.1.5.3 Quale percentuale ha senso calcolare?

Dal punto di vista del calcolo, entrambe le percentuali sono sempre ottenibili, e R le restituisce senza alcun problema. La scelta di quale usare dipende dalla **domanda di ricerca**, cioè da quale variabile ha senso considerare come condizionante e quale come variabile da descrivere.

In questo esempio è più naturale ragionare in termini di $P(\text{Occupazione} \mid \text{Istruzione})$, cioè con le percentuali di **colonna**. Il titolo di studio, infatti, si acquisisce di norma *prima* di entrare nel mondo del lavoro: ha senso chiedersi se e come l'occupazione cambi al variare del titolo di studio, mentre ha molto meno senso pensare che essere casalinga o lavoratrice determini il titolo di studio posseduto. Le percentuali di riga restano comunque utili a scopo descrittivo (per sapere, ad esempio, com'è composto il gruppo delle casalinghe), ma non vanno lette come se l'occupazione "spiegasse" l'istruzione.

Una regola pratica

Si condiziona sulla variabile che si pensa possa *spiegare* o *precedere* l'altra, e si calcolano le percentuali all'interno delle sue categorie. Poi si confrontano le percentuali **tra** le categorie della variabile condizionante: se le distribuzioni sono diverse tra loro, le due variabili sono in qualche modo associate.

Condizionare non vuol dire dimostrare una causa

Scegliere una variabile come condizionante è una scelta di lettura dei dati, non una prova di un legame causale. Che tra le donne con istruzione alta ci siano più lavoratrici non dimostra che sia il titolo di studio a *causare* l'occupazione: le due variabili potrebbero dipendere entrambe da altri fattori, ad esempio la condizione economica della famiglia di origine.

Infine, se le due variabili fossero **indipendenti**, le distribuzioni condizionate sarebbero tutte uguali tra loro e uguali alla distribuzione marginale. In questo caso, ad esempio, in ogni livello di istruzione ci si aspetterebbe di trovare la stessa percentuale di lavoratrici osservata nel campione complessivo, cioè $55/140 \approx 39\%$. Le percentuali osservate (20%, 38% e 60%) sono invece molto